Retail Stocking Priority Classification: Logistic Regression vs Custom Deep Learning vs Hugging Face

Project reframing

This notebook reframes the original retail demand forecasting problem into a **multi-class product stocking priority classification** problem.

Instead of asking only:

- *How many units will sell?*

we also ask:

- *Which products or categories deserve higher stocking priority?*
- *Which items are likely to generate better profit relative to shelf space?*
- *Which items should be deprioritized because of short shelf life, low margin, or supply risk?*

This reframing is closer to how retail managers think in practice.

Modeling goals

We will build and compare three model families:

1. **Logistic Regression** for multi-class classification
2. A **custom deep learning architecture**
3. A **Hugging Face pre-trained transformer** fine-tuned for classification

We will also compare:

- **text-only classification**
- **numeric-only classification**
- **hybrid text + numeric classification**

Business target

We create a derived target called **Stocking Priority Class** with three classes:

- `Low`
- `Medium`
- `High`

This class is based on a business-oriented score that combines:

- profit density
- sales demand
- shelf life
- fill rate
- supplier delay
- backorder risk

Important note

The target used in this notebook is **derived from business rules**, not directly provided in the dataset. That means the models are learning to approximate a business prioritization rule, not a historical label already present in the data.

Mathematical formulation

We define the following intermediate quantities:

1. Unit Margin

$$\text{Unit Margin} = \text{Current Price} - \text{Unit Cost}$$

2. Gross Profit

$$\text{Gross Profit} = \text{Unit Margin} \times \text{Daily Units Sold}$$

3. Profit Density

$$\text{Profit Density} = \frac{\text{Gross Profit}}{\text{Shelf Capacity}}$$

4. Stocking Priority Score

After min-max normalization of relevant variables, we define:

$$\text{Priority Score} = 0.40 \cdot \text{Norm(Profit Density)} + 0.20 \cdot \text{Norm(Daily Units Sold)} + 0.15 \cdot \text{Norm(Fill Rate)} + 0.10 \cdot \text{Norm}$$

Finally, the score is split into three quantile-based classes:

- Low
- Medium
- High

Colab setup

If you are running this notebook in Google Colab and any of the NLP libraries are missing, uncomment and run the next cell once, then restart the runtime.

```
# Optional Colab setup cell
```

```python
# Uncomment only if your environment does not already have these
packages.

# !pip install -q transformers datasets accelerate sentencepiece
evaluate


# ============================================================
# Imports
# ============================================================

import os
import random
import warnings
import inspect

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)

warnings.filterwarnings("ignore")
plt.rcParams["figure.figsize"] = (10, 5)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
from scipy.special import softmax
```

Load the dataset

```python
# ============================================================
# Load dataset
# ============================================================
```

```
df = pd.read_csv("/content/master_retail_dataset_v2.csv")

print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset shape: (2000, 58)
```

{"type":"dataframe"}

Inspect columns and data types

```
# Basic inspection
print("\nColumns:")
print(df.columns.tolist())

print("\nData types:")
display(df.dtypes)
```

```
Columns:
['Date', 'DayOfWeek', 'Month', 'Year', 'Is_Weekend', 'Is_Holiday',
'Product_ID', 'Category', 'Base_Price', 'Discount_Percentage',
'Current_Price', 'Store_ID', 'Store_Type', 'Competitor_Price',
'Footfall_Index', 'Avg_Temperature', 'Rainfall_mm',
'Social_Media_Sentiment', 'Lead_Time_Days', 'Safety_Stock_Level',
'Stock_On_Hand', 'Daily_Units_Sold', 'Festival_Name', 'Festival_Type',
'UPC_EAN', 'Product_Name', 'Sub_Category', 'Brand_Name', 'Brand_Tier',
'Shelf_Life_Days', 'No_of_Checkout_Counters', 'Avg_Billing_Time_min',
'Local_Population_Density', 'Product_Age_Days',
'No_of_Customer_Purchases', 'Unit_Cost', 'Promotional_Campaign_Flag',
'Competitor_Promotion_Flag', 'Google_Trends_Current_Wk',
'Google_Trends_Lag_1w', 'Shelf_Capacity', 'Promotion_Type',
'Marketing_Spend', 'Payday_Flag', 'School_Vacation_Flag',
'Local_Event_Flag', 'Launch_Date', 'Seasonal_Product_Flag',
'Online_Sales_Units', 'In_Store_Sales_Units', 'App_Traffic_Index',
'Website_Visits', 'Backorder_Flag', 'Supplier_Delay_Days',
'Fill_Rate_Pct', 'Loyalty_Program_Usage_Count',
'Repeat_Purchase_Rate', 'Avg_Basket_Size']

Data types:

Date                              object
DayOfWeek                          int64
Month                              int64
Year                               int64
Is_Weekend                         int64
Is_Holiday                         int64
Product_ID                         int64
Category                          object
```

```
Base_Price                      float64
Discount_Percentage             float64
Current_Price                   float64
Store_ID                          int64
Store_Type                       object
Competitor_Price                float64
Footfall_Index                  float64
Avg_Temperature                 float64
Rainfall_mm                     float64
Social_Media_Sentiment          float64
Lead_Time_Days                    int64
Safety_Stock_Level                int64
Stock_On_Hand                     int64
Daily_Units_Sold                  int64
Festival_Name                    object
Festival_Type                    object
UPC_EAN                           int64
Product_Name                     object
Sub_Category                     object
Brand_Name                       object
Brand_Tier                       object
Shelf_Life_Days                   int64
No_of_Checkout_Counters           int64
Avg_Billing_Time_min            float64
Local_Population_Density          int64
Product_Age_Days                  int64
No_of_Customer_Purchases          int64
Unit_Cost                       float64
Promotional_Campaign_Flag        object
Competitor_Promotion_Flag        object
Google_Trends_Current_Wk          int64
Google_Trends_Lag_1w              int64
Shelf_Capacity                    int64
Promotion_Type                   object
Marketing_Spend                   int64
Payday_Flag                       int64
School_Vacation_Flag              int64
Local_Event_Flag                  int64
Launch_Date                      object
Seasonal_Product_Flag            object
Online_Sales_Units                int64
In_Store_Sales_Units              int64
App_Traffic_Index                 int64
Website_Visits                    int64
Backorder_Flag                   object
Supplier_Delay_Days               int64
Fill_Rate_Pct                   float64
Loyalty_Program_Usage_Count       int64
Repeat_Purchase_Rate            float64
```

```
Avg_Basket_Size                 float64
dtype: object
```

Clean selected flag columns

Some business flags are stored as text. We convert them into numeric 0/1 form where needed.

```
# ============================================================
# Clean selected binary-like flags
# ============================================================

binary_map = {
    "yes": 1, "y": 1, "true": 1, "1": 1,
    "no": 0, "n": 0, "false": 0, "0": 0
}

for col in ["Backorder_Flag", "Promotional_Campaign_Flag",
"Competitor_Promotion_Flag", "Seasonal_Product_Flag"]:
    if col in df.columns:
        df[col] = (
            df[col]
            .astype(str)
            .str.strip()
            .str.lower()
            .map(binary_map)
            .fillna(0)
            .astype(int)
        )

# Convert launch date to datetime
df["Launch_Date"] = pd.to_datetime(df["Launch_Date"], errors="coerce")
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

display(df[["Backorder_Flag", "Promotional_Campaign_Flag",
"Competitor_Promotion_Flag", "Seasonal_Product_Flag"]].head())
```


```
{"summary":"{\n  \"name\": \"display(df[[\\\"Backorder_Flag\\\",
\\\"Promotional_Campaign_Flag\\\", \\\"Competitor_Promotion_Flag\\\",
\\\"Seasonal_Product_Flag\\\"]]\",\n  \"rows\": 5,\n  \"fields\": [\n
{\n      \"column\": \"Backorder_Flag\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 0,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Promotional_Campaign_Flag\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1\n        ],\n        \"semantic_type\": \"\",\n
```

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Competitor_Promotion_Flag\",\n   \"properties\": {\n \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   1\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Seasonal_Product_Flag\",\n   \"properties\": {\n \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Create the business target: Stocking Priority Class

```
# ============================================================
# Create business-oriented target
# ============================================================

# 1. Unit margin
df["Unit_Margin"] = df["Current_Price"] - df["Unit_Cost"]

# 2. Gross profit
df["Gross_Profit"] = df["Unit_Margin"] * df["Daily_Units_Sold"]

# 3. Profit density relative to shelf capacity
df["Profit_Density"] = df["Gross_Profit"] /
df["Shelf_Capacity"].replace(0, np.nan)
df["Profit_Density"] = df["Profit_Density"].fillna(0)

# Helper function for min-max normalization
def minmax_scale(series):
    series = series.astype(float)
    mn = series.min()
    mx = series.max()
    if mx == mn:
        return pd.Series(np.zeros(len(series)), index=series.index)
    return (series - mn) / (mx - mn)

# Normalized components
df["Norm_Profit_Density"] = minmax_scale(df["Profit_Density"])
df["Norm_Daily_Units_Sold"] = minmax_scale(df["Daily_Units_Sold"])
df["Norm_Fill_Rate"] = minmax_scale(df["Fill_Rate_Pct"])
df["Norm_Shelf_Life"] = minmax_scale(df["Shelf_Life_Days"])
df["Norm_Supplier_Delay"] = minmax_scale(df["Supplier_Delay_Days"])

# Business-oriented priority score
df["Stocking_Priority_Score"] = (
    0.40 * df["Norm_Profit_Density"]
    + 0.20 * df["Norm_Daily_Units_Sold"]
```

```
    + 0.15 * df["Norm_Fill_Rate"]
    + 0.10 * df["Norm_Shelf_Life"]
    + 0.10 * (1 - df["Norm_Supplier_Delay"])
    + 0.05 * (1 - df["Backorder_Flag"])
)

# Convert score into 3 balanced classes
df["Stocking_Priority_Class"] = pd.qcut(
    df["Stocking_Priority_Score"],
    q=3,
    labels=["Low", "Medium", "High"]
)

print("Class distribution:")
display(df["Stocking_Priority_Class"].value_counts())
```

```
Class distribution:

Stocking_Priority_Class
Low       667
High      667
Medium    666
Name: count, dtype: int64
```

Initial business analysis

The next cells generate actual analysis outputs for:

1. Which categories appear most often in the **High** stocking priority class
2. Which categories have the shortest average shelf life
3. Which products rank highest on the stocking-priority score

```
# ============================================================
# Category-level analysis
# ============================================================

category_summary = (
    df.groupby("Category")
      .agg(
          Records=("Category", "size"),
          Avg_Daily_Units_Sold=("Daily_Units_Sold", "mean"),
          Avg_Current_Price=("Current_Price", "mean"),
          Avg_Unit_Margin=("Unit_Margin", "mean"),
          Avg_Gross_Profit=("Gross_Profit", "mean"),
          Avg_Shelf_Capacity=("Shelf_Capacity", "mean"),
          Avg_Shelf_Life_Days=("Shelf_Life_Days", "mean")
      )
)
```

```
category_high_share = (
    df.assign(Is_High=(df["Stocking_Priority_Class"] ==
"High").astype(int))
      .groupby("Category")["Is_High"]
      .mean()
      .rename("High_Priority_Share")
)

category_summary = category_summary.join(category_high_share)
category_summary = category_summary.sort_values("High_Priority_Share",
ascending=False)

display(category_summary.round(3))
```


```
{"summary":"{\n  \"name\": \"display(category_summary\",\n  \"rows\":
5,\n  \"fields\": [\n    {\n      \"column\": \"Category\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"Clothing\",\n          \"Grocery\",\n          \"Electronics\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Records\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
25,\n        \"min\": 378,\n        \"max\": 442,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          378,\n
442,\n          387\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Avg_Daily_Units_Sold\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.1428570339285693,\n        \"min\":
110.687,\n        \"max\": 113.405,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          113.257,\n          113.405,\n
110.687\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Avg_Current_Price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5.68180836530061,\n        \"min\":
180.041,\n        \"max\": 194.809,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          184.61,\n          180.041,\n
194.809\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Avg_Unit_Margin\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.4673706757326181,\n        \"min\":
16.219,\n        \"max\": 20.036,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          16.219,\n          17.507,\n
18.332\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Avg_Gross_Profit\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 149.3905292306042,\n        \"min\":
1340.999,\n        \"max\": 1690.405,\n        \"num_unique_values\":
5,\n        \"samples\": [\n          1340.999,\n          1577.671,\n
1412.922\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

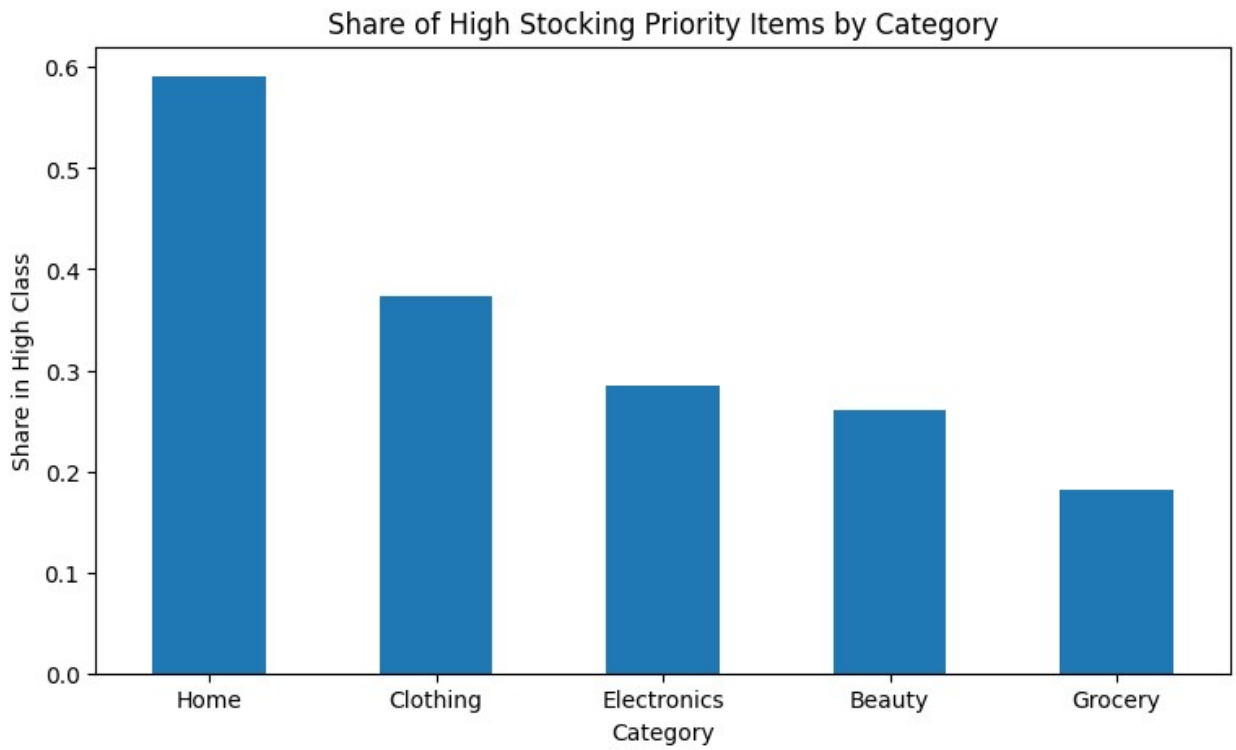
\"Avg_Shelf_Capacity\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1.8675471346126675,\n \"min\": 68.888,\n \"max\": 73.829,\n \"num_unique_values\": 5,\n \"samples\": [\n 69.765,\n 71.068,\n 73.829\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Avg_Shelf_Life_Days\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 834.4495518955595,\n \"min\": 42.649,\n \"max\": 2222.056,\n \"num_unique_values\": 5,\n \"samples\": [\n 943.532,\n 42.649,\n 1394.491\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"High_Priority_Share\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.15710506038953678,\n \"min\": 0.181,\n \"max\": 0.591,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.373,\n 0.181,\n 0.284\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

Interpretation

The table above identifies which categories are more likely to appear in the **High** stocking priority class under the business rule defined in this notebook.

In this dataset, categories such as **Home**, **Clothing**, **Electronics**, and **Beauty** tend to perform better than **Grocery** on the stocking-priority score. Grocery remains important for sales, but its much shorter shelf life can reduce its stocking attractiveness under this business framing.

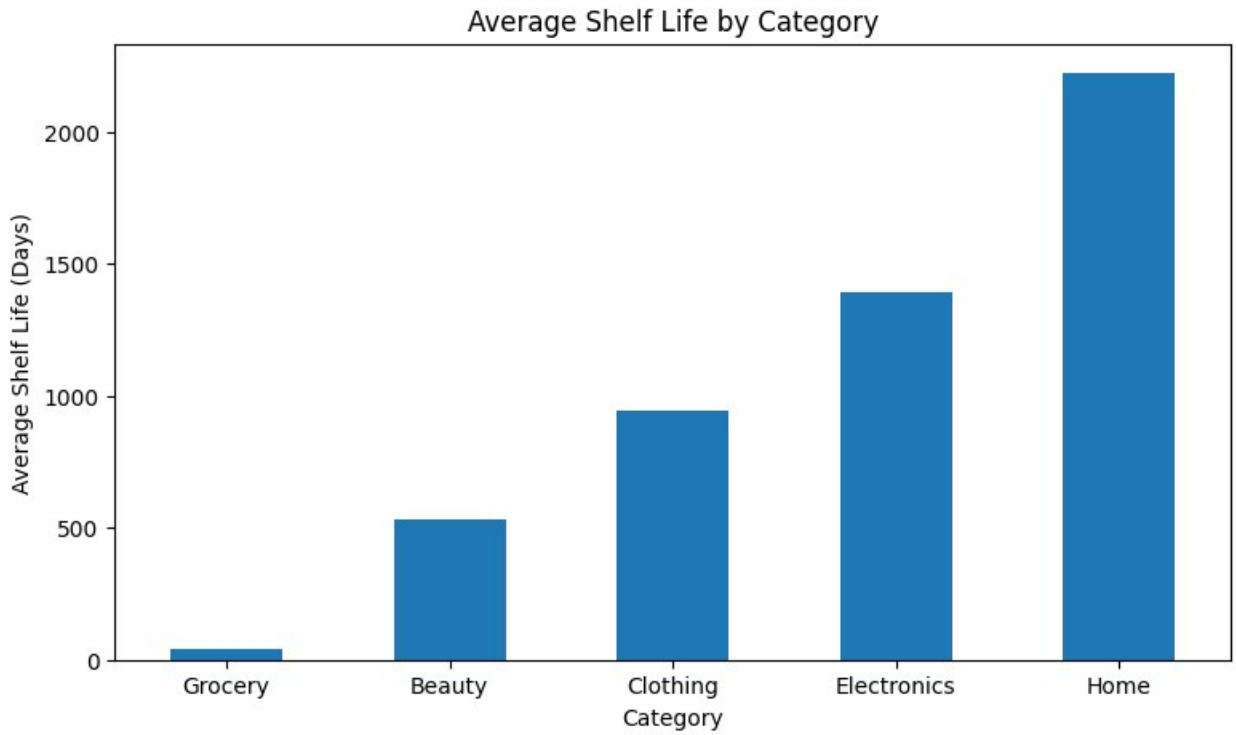
```
# Plot the share of High priority items by category
high_share_plot =
category_summary["High_Priority_Share"].sort_values(ascending=False)

plt.figure(figsize=(9, 5))
high_share_plot.plot(kind="bar")
plt.title("Share of High Stocking Priority Items by Category")
plt.ylabel("Share in High Class")
plt.xlabel("Category")
plt.xticks(rotation=0)
plt.show()
```




```python
# ============================================================
# Shelf-life analysis
# ============================================================

shelf_life_by_category = (
    df.groupby("Category")["Shelf_Life_Days"]
      .mean()
      .sort_values()
)

display(shelf_life_by_category.round(2))

plt.figure(figsize=(9, 5))
shelf_life_by_category.plot(kind="bar")
plt.title("Average Shelf Life by Category")
plt.ylabel("Average Shelf Life (Days)")
plt.xlabel("Category")
plt.xticks(rotation=0)
plt.show()
```

```
Category
Grocery          42.65
Beauty          530.91
Clothing        943.53
Electronics    1394.49
```

```
Home           2222.06
Name: Shelf_Life_Days, dtype: float64
```



Interpretation

Shelf life is central to the instructor's framing.

If a category has short shelf life, heavy stocking can create spoilage risk and reduce business attractiveness even when demand is strong. In this dataset, **Grocery** has by far the shortest average shelf life, which makes it more operationally risky than categories such as **Home** or **Electronics**.

```
# ============================================================
# Top products by average stocking priority score
# ============================================================

top_products = (
    df.groupby(["Category", "Product_Name"])
      .agg(
          Avg_Priority_Score=("Stocking_Priority_Score", "mean"),
          Avg_Profit_Density=("Profit_Density", "mean"),
          Avg_Shelf_Life=("Shelf_Life_Days", "mean"),
          Records=("Product_Name", "size")
      )
      .sort_values("Avg_Priority_Score", ascending=False)
```

```
      .head(15)
)

display(top_products.round(3))
```

{"summary":"{\n \"name\": \"display(top_products\",\n \"rows\": 15,\n \"fields\": [\n {\n \"column\": \"Avg_Priority_Score\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.011907180706655654,\n \"min\": 0.524,\n \"max\": 0.561,\n \"num_unique_values\": 13,\n \"samples\": [\n 0.528,\n 0.533,\n 0.561\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Avg_Profit_Density\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 22.294702473479038,\n \"min\": -14.767,\n \"max\": 63.025,\n \"num_unique_values\": 15,\n \"samples\": [\n 51.706,\n 16.206,\n 27.981\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Avg_Shelf_Life\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 555.7844477681545,\n \"min\": 1057.0,\n \"max\": 2850.0,\n \"num_unique_values\": 15,\n \"samples\": [\n 2326.0,\n 2178.0,\n 2366.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Records\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 4,\n \"min\": 11,\n \"max\": 25,\n \"num_unique_values\": 9,\n \"samples\": [\n 25,\n 21,\n 15\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

Build a column-aware text template

Instead of using a raw combined text field, we now build a more meaningful text template that preserves the role of each textual attribute.

Example template

`Category: Beauty | Product: Lakme Makeup 1009 | Brand: Lakme | Brand Tier: Premium | Promotion: Flash Sale | Festival: Diwali`

This is more informative than simply concatenating tokens without context.

```
# ============================================================
# Create column-aware text template
# ============================================================

text_columns_for_template = [
```

```
    "Category", "Product_Name", "Sub_Category", "Brand_Name",
    "Brand_Tier", "Store_Type", "Promotion_Type", "Festival_Name",
    "Festival_Type"
]

for col in text_columns_for_template:
    df[col] = df[col].fillna("None").astype(str)

df["text_template"] = (
    "Category: " + df["Category"]
    + " | Product: " + df["Product_Name"]
    + " | SubCategory: " + df["Sub_Category"]
    + " | Brand: " + df["Brand_Name"]
    + " | BrandTier: " + df["Brand_Tier"]
    + " | StoreType: " + df["Store_Type"]
    + " | Promotion: " + df["Promotion_Type"]
    + " | FestivalName: " + df["Festival_Name"]
    + " | FestivalType: " + df["Festival_Type"]
)

display(df[["text_template"]].head(3))
```

{"summary":"{\n \"name\": \"display(df[[\\\"text_template\\\"]]\",\n \"rows\": 3,\n \"fields\": [\n {\n \"column\": \"text_template\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 3,\n \"samples\": [\n \"Category: Grocery | Product: Nestle Beverages 1002 | SubCategory: Beverages | Brand: Nestle | BrandTier: Premium | StoreType: Urban | Promotion: None | FestivalName: None | FestivalType: None\",\n \"Category: Home | Product: Tupperware Decor 1014 | SubCategory: Decor | Brand: Tupperware | BrandTier: Budget | StoreType: Rural | Promotion: Flash Sale | FestivalName: None | FestivalType: None\",\n \"Category: Electronics | Product: Sony Computers 1015 | SubCategory: Computers | Brand: Sony | BrandTier: Mid | StoreType: Urban | Promotion: None | FestivalName: None | FestivalType: None\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

Define label encoder and train/validation/test splits

We use a **stratified** split so that the class balance is preserved.

This is important because the task is now **multi-class classification**.

```
# ============================================================
# Encode target labels
# ============================================================
```

```
label_encoder = LabelEncoder()
df["label_id"] =
label_encoder.fit_transform(df["Stocking_Priority_Class"])

# Build Python-native label dictionaries for compatibility with
# Hugging Face model configuration (JSON serializable keys/values).
class_names = [str(x) for x in label_encoder.classes_]
label_mapping = {str(label): int(i) for i, label in
enumerate(class_names)}
inverse_label_mapping = {int(i): str(label) for i, label in
enumerate(class_names)}

print("Label mapping:", label_mapping)
print("Inverse label mapping:", inverse_label_mapping)
```

```
Label mapping: {'High': 0, 'Low': 1, 'Medium': 2}
Inverse label mapping: {0: 'High', 1: 'Low', 2: 'Medium'}
```

```
# ============================================================
# Train / validation / test split
# ============================================================

train_df, temp_df = train_test_split(
    df,
    test_size=0.30,
    random_state=SEED,
    stratify=df["label_id"]
)

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=SEED,
    stratify=temp_df["label_id"]
)

print("Train shape:", train_df.shape)
print("Validation shape:", val_df.shape)
print("Test shape:", test_df.shape)

print("\nTrain class distribution:")
display(train_df["Stocking_Priority_Class"].value_counts(normalize=Tru
e).round(3))

print("\nValidation class distribution:")
display(val_df["Stocking_Priority_Class"].value_counts(normalize=True)
.round(3))

print("\nTest class distribution:")
```

```
display(test_df["Stocking_Priority_Class"].value_counts(normalize=True
).round(3))
```

```
Train shape: (1400, 70)
Validation shape: (300, 70)
Test shape: (300, 70)

Train class distribution:

Stocking_Priority_Class
Low       0.334
High      0.334
Medium    0.333
Name: proportion, dtype: float64


Validation class distribution:

Stocking_Priority_Class
Low       0.333
Medium    0.333
High      0.333
Name: proportion, dtype: float64


Test class distribution:

Stocking_Priority_Class
Low       0.333
Medium    0.333
High      0.333
Name: proportion, dtype: float64
```

Prepare numeric features

To avoid trivial leakage, we **exclude** the exact variables used to construct the target score whenever possible.

We keep context-rich numeric variables such as:

- traffic
- seasonality proxies
- marketing
- store operations
- trends
- customer behavior

```
# ============================================================
# Numeric feature selection
```

```python
# ============================================================

numeric_feature_cols = [
    "DayOfWeek",
    "Month",
    "Year",
    "Is_Weekend",
    "Is_Holiday",
    "Base_Price",
    "Discount_Percentage",
    "Competitor_Price",
    "Footfall_Index",
    "Avg_Temperature",
    "Rainfall_mm",
    "Lead_Time_Days",
    "Safety_Stock_Level",
    "Stock_On_Hand",
    "No_of_Checkout_Counters",
    "Avg_Billing_Time_min",
    "Local_Population_Density",
    "Product_Age_Days",
    "No_of_Customer_Purchases",
    "Promotional_Campaign_Flag",
    "Competitor_Promotion_Flag",
    "Google_Trends_Current_Wk",
    "Google_Trends_Lag_1w",
    "Marketing_Spend",
    "Payday_Flag",
    "School_Vacation_Flag",
    "Local_Event_Flag",
    "Seasonal_Product_Flag",
    "App_Traffic_Index",
    "Website_Visits",
    "Loyalty_Program_Usage_Count",
    "Repeat_Purchase_Rate",
    "Avg_Basket_Size"
]

# Ensure the selected columns exist
numeric_feature_cols = [col for col in numeric_feature_cols if col in
df.columns]

print("Numeric features used:", len(numeric_feature_cols))
print(numeric_feature_cols)
```

```
Numeric features used: 33
['DayOfWeek', 'Month', 'Year', 'Is_Weekend', 'Is_Holiday',
'Base_Price', 'Discount_Percentage', 'Competitor_Price',
'Footfall_Index', 'Avg_Temperature', 'Rainfall_mm', 'Lead_Time_Days',
'Safety_Stock_Level', 'Stock_On_Hand', 'No_of_Checkout_Counters',
```

```
'Avg_Billing_Time_min', 'Local_Population_Density',
'Product_Age_Days', 'No_of_Customer_Purchases',
'Promotional_Campaign_Flag', 'Competitor_Promotion_Flag',
'Google_Trends_Current_Wk', 'Google_Trends_Lag_1w', 'Marketing_Spend',
'Payday_Flag', 'School_Vacation_Flag', 'Local_Event_Flag',
'Seasonal_Product_Flag', 'App_Traffic_Index', 'Website_Visits',
'Loyalty_Program_Usage_Count', 'Repeat_Purchase_Rate',
'Avg_Basket_Size']
```

1. Logistic Regression Baselines

We compare three variants:

1. **Text-only logistic regression**
2. **Numeric-only logistic regression**
3. **Hybrid logistic regression (text + numeric)**

This helps us understand whether the business signal is coming more from category/product text, structured numeric context, or both together.

```
# ============================================================
# Helper functions for evaluation
# ============================================================

def evaluate_predictions(y_true, y_pred, model_name):
    """
    Return a dictionary with common multi-class classification
metrics.
    """
    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Macro_F1": f1_score(y_true, y_pred, average="macro"),
        "Weighted_F1": f1_score(y_true, y_pred, average="weighted")
    }

def print_report(y_true, y_pred, title):
    print(f"\n{title}")
    print("=" * len(title))
    print(classification_report(y_true, y_pred,
target_names=label_encoder.classes_))


# ============================================================
# Text-only logistic regression
# ============================================================
```

```
text_only_clf = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=5000, ngram_range=(1, 2))),
    ("clf", LogisticRegression(max_iter=3000, class_weight="balanced",
random_state=SEED))
])

text_only_clf.fit(train_df["text_template"], train_df["label_id"])
text_only_pred = text_only_clf.predict(test_df["text_template"])

text_only_metrics = evaluate_predictions(test_df["label_id"],
text_only_pred, "LogReg_TextOnly")
print_report(test_df["label_id"], text_only_pred, "Text-only Logistic
Regression Report")
text_only_metrics
```

```
Text-only Logistic Regression Report
====================================
              precision    recall  f1-score   support

        High       0.52      0.49      0.50       100
         Low       0.42      0.56      0.48       100
      Medium       0.36      0.26      0.30       100

    accuracy                           0.44       300
   macro avg       0.43      0.44      0.43       300
weighted avg       0.43      0.44      0.43       300
```

```
{'Model': 'LogReg_TextOnly',
 'Accuracy': 0.43666666666666665,
 'Macro_F1': 0.4285254597461405,
 'Weighted_F1': 0.4285254597461404}
```

```
# ============================================================
# Numeric-only logistic regression
# ============================================================

numeric_only_preprocessor = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

numeric_only_X_train =
numeric_only_preprocessor.fit_transform(train_df[numeric_feature_cols]
)
numeric_only_X_test =
numeric_only_preprocessor.transform(test_df[numeric_feature_cols])

numeric_only_clf = LogisticRegression(
```

```
    max_iter=3000,
    class_weight="balanced",
    random_state=SEED
)

numeric_only_clf.fit(numeric_only_X_train, train_df["label_id"])
numeric_only_pred = numeric_only_clf.predict(numeric_only_X_test)

numeric_only_metrics = evaluate_predictions(test_df["label_id"],
numeric_only_pred, "LogReg_NumericOnly")
print_report(test_df["label_id"], numeric_only_pred, "Numeric-only
Logistic Regression Report")
numeric_only_metrics
```

```
Numeric-only Logistic Regression Report
========================================
              precision    recall  f1-score   support

        High       0.44      0.51      0.47       100
         Low       0.45      0.56      0.50       100
      Medium       0.35      0.21      0.26       100

    accuracy                           0.43       300
   macro avg       0.41      0.43      0.41       300
weighted avg       0.41      0.43      0.41       300
```

```
{'Model': 'LogReg_NumericOnly',
 'Accuracy': 0.4266666666666667,
 'Macro_F1': 0.4115740740740741,
 'Weighted_F1': 0.4115740740740741}
```

```
# ============================================================
# Hybrid logistic regression (text + numeric)
# ============================================================

hybrid_preprocessor = ColumnTransformer([
    ("text", TfidfVectorizer(max_features=5000, ngram_range=(1, 2)),
"text_template"),
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_feature_cols)
])

hybrid_clf = Pipeline([
    ("preprocessor", hybrid_preprocessor),
    ("clf", LogisticRegression(max_iter=3000, class_weight="balanced",
random_state=SEED))
```

```
])

hybrid_clf.fit(train_df[["text_template"] + numeric_feature_cols],
train_df["label_id"])
hybrid_pred = hybrid_clf.predict(test_df[["text_template"] +
numeric_feature_cols])

hybrid_metrics = evaluate_predictions(test_df["label_id"],
hybrid_pred, "LogReg_Hybrid")
print_report(test_df["label_id"], hybrid_pred, "Hybrid Logistic
Regression Report")
hybrid_metrics
```

```
Hybrid Logistic Regression Report
==================================
              precision    recall  f1-score   support

        High       0.50      0.55      0.53       100
         Low       0.54      0.58      0.56       100
      Medium       0.35      0.29      0.32       100

    accuracy                           0.47       300
   macro avg       0.46      0.47      0.47       300
weighted avg       0.46      0.47      0.47       300
```

```
{'Model': 'LogReg_Hybrid',
 'Accuracy': 0.4733333333333333,
 'Macro_F1': 0.4669826626254581,
 'Weighted_F1': 0.4669826626254581}
```

```
# Logistic regression comparison table
logreg_results = pd.DataFrame([
    text_only_metrics,
    numeric_only_metrics,
    hybrid_metrics
]).sort_values(["Weighted_F1", "Accuracy"], ascending=False)

display(logreg_results.round(4))
```

{"summary":"{\n \"name\": \"display(logreg_results\",\n \"rows\": 3,\n \"fields\": [\n {\n \"column\": \"Model\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 3,\n \"samples\": [\n \"LogReg_Hybrid\",\n \"LogReg_TextOnly\",\n \"LogReg_NumericOnly\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Accuracy\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.024532699267168563,\n \"min\":

```
0.4267,\n        \"max\": 0.4733,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.4733,\n          0.4367,\n
0.4267\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Macro_F1\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.02839313297260449,\n        \"min\":
0.4116,\n        \"max\": 0.467,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.467,\n          0.4285,\n          0.4116\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Weighted_F1\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.02839313297260449,\n        \"min\":
0.4116,\n        \"max\": 0.467,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.467,\n          0.4285,\n          0.4116\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Interpretation

The logistic regression comparison tells us whether:

- product/category text alone is enough
- numeric context alone is enough
- or the **hybrid view** is strongest

In many retail problems, the hybrid setup performs best because it combines:

- semantic category/product information
- structured operational context

2. Custom Deep Learning Architecture

We build a custom **hybrid neural network** with:

- a **text branch** that learns from the column-aware text template
- a **numeric branch** that learns from structured numeric features

The two branches are merged before the final softmax classification layer.

```
# ============================================================
# TensorFlow / Keras imports
# ============================================================

import tensorflow as tf
from tensorflow.keras.layers import (
    Input,
    TextVectorization,
```

```
    Embedding,
    Bidirectional,
    LSTM,
    Dense,
    Dropout,
    Concatenate
)
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping

tf.random.set_seed(SEED)


# ==============================================================
# Prepare data for the custom hybrid model
# ==============================================================

# Text arrays
X_train_text = train_df["text_template"].astype(str).values
X_val_text = val_df["text_template"].astype(str).values
X_test_text = test_df["text_template"].astype(str).values

# Numeric arrays
numeric_imputer = SimpleImputer(strategy="median")
numeric_scaler = StandardScaler()

X_train_num =
numeric_imputer.fit_transform(train_df[numeric_feature_cols])
X_val_num = numeric_imputer.transform(val_df[numeric_feature_cols])
X_test_num = numeric_imputer.transform(test_df[numeric_feature_cols])

X_train_num = numeric_scaler.fit_transform(X_train_num)
X_val_num = numeric_scaler.transform(X_val_num)
X_test_num = numeric_scaler.transform(X_test_num)

# Labels
y_train = train_df["label_id"].values
y_val = val_df["label_id"].values
y_test = test_df["label_id"].values

print("Text training shape:", X_train_text.shape)
print("Numeric training shape:", X_train_num.shape)
```

```
Text training shape: (1400,)
Numeric training shape: (1400, 33)
```

```
# ==============================================================
# Text vectorization layer
# ==============================================================
```

```python
max_tokens = 8000
sequence_length = 40

text_vectorizer = TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=sequence_length
)

# Adapt vectorizer only on training text
text_vectorizer.adapt(X_train_text)


# ============================================================
# Build custom hybrid neural network
# ============================================================

# Text input branch
text_input = Input(shape=(1,), dtype=tf.string, name="text_input")
x_text = text_vectorizer(text_input)
x_text = Embedding(input_dim=max_tokens, output_dim=64)(x_text)
x_text = Bidirectional(LSTM(32))(x_text)
x_text = Dense(32, activation="relu")(x_text)
x_text = Dropout(0.3)(x_text)

# Numeric input branch
num_input = Input(shape=(X_train_num.shape[1],), name="numeric_input")
x_num = Dense(64, activation="relu")(num_input)
x_num = Dropout(0.3)(x_num)
x_num = Dense(32, activation="relu")(x_num)

# Merge both branches
x = Concatenate()([x_text, x_num])
x = Dense(64, activation="relu")(x)
x = Dropout(0.3)(x)
output = Dense(len(label_encoder.classes_), activation="softmax")(x)

custom_model = Model(inputs=[text_input, num_input], outputs=output)

custom_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

custom_model.summary()
```

```
Model: "functional"
```

```
┏━━━
```

```
┃ Layer (type)        ┃ Output Shape      ┃    Param # ┃ Connected to
┃                     ┃                   ┃            ┃
━━━
```

```
──────┤
│ dense_2 (Dense)      │ (None, 32)         │       2,080 │ dropout_1[0]
[0]    │
├────────────────────────┼────────────────────┼─────────────┼────────────────
──────┤
│ concatenate          │ (None, 64)         │           0 │ dropout[0]
[0],    │
│ (Concatenate)        │                    │             │ dense_2[0][0]
│
├────────────────────────┼────────────────────┼─────────────┼────────────────
──────┤
│ dense_3 (Dense)      │ (None, 64)         │       4,160 │
concatenate[0][0] │
├────────────────────────┼────────────────────┼─────────────┼────────────────
──────┤
│ dropout_2 (Dropout) │ (None, 64)         │           0 │ dense_3[0][0]
│
├────────────────────────┼────────────────────┼─────────────┼────────────────
──────┤
│ dense_4 (Dense)      │ (None, 3)          │         195 │ dropout_2[0]
[0]    │
└────────────────────────┴────────────────────┴─────────────┴────────────────
──────┘

 Total params: 547,523 (2.09 MB)

 Trainable params: 547,523 (2.09 MB)

 Non-trainable params: 0 (0.00 B)
```

```python
# ============================================================
# Train custom hybrid neural network
# ============================================================

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history = custom_model.fit(
    {"text_input": X_train_text, "numeric_input": X_train_num},
    y_train,
    validation_data=(
        {"text_input": X_val_text, "numeric_input": X_val_num},
        y_val
    ),
    epochs=8,
    batch_size=32,
```

```
    callbacks=[early_stop],
    verbose=1
)
```

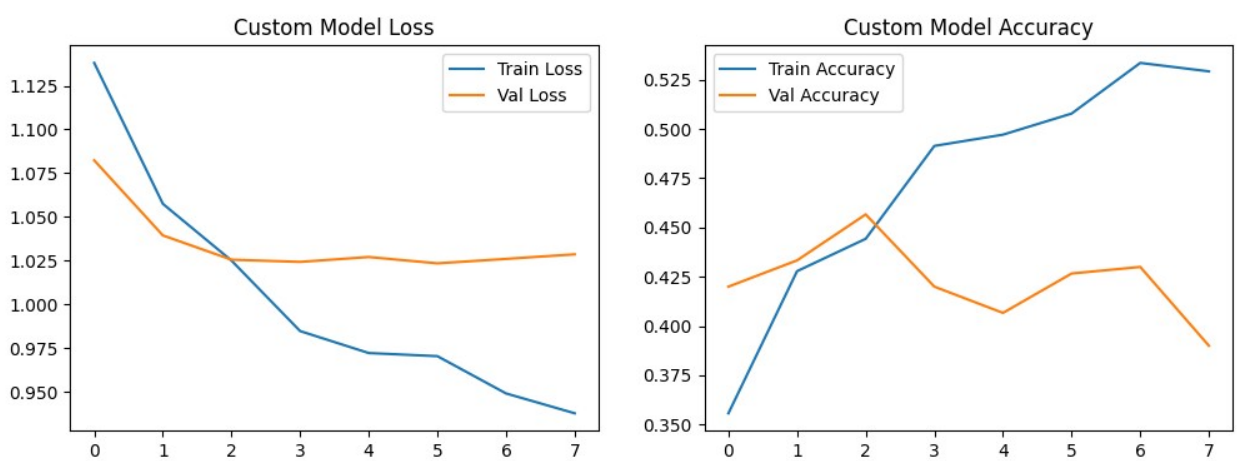
```
Epoch 1/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 18s 158ms/step - accuracy: 0.3557 - loss:
1.1380 - val_accuracy: 0.4200 - val_loss: 1.0824
Epoch 2/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 4s 80ms/step - accuracy: 0.4279 - loss:
1.0575 - val_accuracy: 0.4333 - val_loss: 1.0395
Epoch 3/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 5s 74ms/step - accuracy: 0.4443 - loss:
1.0251 - val_accuracy: 0.4567 - val_loss: 1.0256
Epoch 4/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 3s 75ms/step - accuracy: 0.4914 - loss:
0.9849 - val_accuracy: 0.4200 - val_loss: 1.0244
Epoch 5/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 6s 91ms/step - accuracy: 0.4971 - loss:
0.9723 - val_accuracy: 0.4067 - val_loss: 1.0272
Epoch 6/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 4s 95ms/step - accuracy: 0.5079 - loss:
0.9705 - val_accuracy: 0.4267 - val_loss: 1.0235
Epoch 7/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 5s 101ms/step - accuracy: 0.5336 - loss:
0.9492 - val_accuracy: 0.4300 - val_loss: 1.0261
Epoch 8/8
44/44 ━━━━━━━━━━━━━━━━━━━━ 4s 86ms/step - accuracy: 0.5293 - loss:
0.9379 - val_accuracy: 0.3900 - val_loss: 1.0287
```

```
# ============================================================
# Evaluate custom hybrid neural network
# ============================================================

custom_pred_probs = custom_model.predict(
    {"text_input": X_test_text, "numeric_input": X_test_num},
    verbose=0
)

custom_pred = np.argmax(custom_pred_probs, axis=1)

custom_metrics = evaluate_predictions(y_test, custom_pred,
"CustomDL_Hybrid")
print_report(y_test, custom_pred, "Custom Deep Learning Hybrid Model
Report")
custom_metrics
```

```
Custom Deep Learning Hybrid Model Report
========================================
```

```
              precision    recall  f1-score   support

        High       0.52      0.62      0.57       100
         Low       0.47      0.63      0.54       100
      Medium       0.43      0.20      0.27       100

    accuracy                           0.48       300
   macro avg       0.47      0.48      0.46       300
weighted avg       0.47      0.48      0.46       300


{'Model': 'CustomDL_Hybrid',
 'Accuracy': 0.48333333333333334,
 'Macro_F1': 0.45892680922035134,
 'Weighted_F1': 0.4589268092203513}
```

```
# Plot training curves
history_df = pd.DataFrame(history.history)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history_df["loss"], label="Train Loss")
axes[0].plot(history_df["val_loss"], label="Val Loss")
axes[0].set_title("Custom Model Loss")
axes[0].legend()

axes[1].plot(history_df["accuracy"], label="Train Accuracy")
axes[1].plot(history_df["val_accuracy"], label="Val Accuracy")
axes[1].set_title("Custom Model Accuracy")
axes[1].legend()

plt.show()
```



3. Hugging Face Pre-trained Transformer

For the Hugging Face baseline, we fine-tune a pre-trained text classification model on the **column-aware text template**.

To keep training practical in Colab, we use a compact transformer such as:

- `distilbert-base-uncased`

This model is **text-only**, so it does not use the numeric feature block.

```
# ============================================================
# Hugging Face imports
# ============================================================

from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    TrainingArguments,
    Trainer
)


# ============================================================
# Prepare Hugging Face datasets
# ============================================================

hf_train_df = train_df[["text_template",
"label_id"]].rename(columns={"text_template": "text", "label_id":
"label"})
hf_val_df = val_df[["text_template",
"label_id"]].rename(columns={"text_template": "text", "label_id":
"label"})
hf_test_df = test_df[["text_template",
"label_id"]].rename(columns={"text_template": "text", "label_id":
"label"})

hf_train = Dataset.from_pandas(hf_train_df.reset_index(drop=True))
hf_val = Dataset.from_pandas(hf_val_df.reset_index(drop=True))
hf_test = Dataset.from_pandas(hf_test_df.reset_index(drop=True))


# ============================================================
# Tokenizer and tokenization
# ============================================================

checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

```
def tokenize_batch(batch):
    return tokenizer(batch["text"], truncation=True)

tokenized_train = hf_train.map(tokenize_batch, batched=True)
tokenized_val = hf_val.map(tokenize_batch, batched=True)
tokenized_test = hf_test.map(tokenize_batch, batched=True)

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"a5dbd0bc8f7348319e67b7c13b158281","version_major":2,"vers
ion_minor":0}

{"model_id":"ee483e63c20940b48a2067961d851215","version_major":2,"vers
ion_minor":0}

{"model_id":"239d062737ff44ad8d893f460e97a6de","version_major":2,"vers
ion_minor":0}

{"model_id":"059c4fdb6dfa49748401605f335daef5","version_major":2,"vers
ion_minor":0}

{"model_id":"4e4d1df207934ccbb32d15635d859cf2","version_major":2,"vers
ion_minor":0}

{"model_id":"e3d54da44a7f4887ab2b27e0de9e506f","version_major":2,"vers
ion_minor":0}

{"model_id":"2bdd42907c104a26ac904edd05f494a8","version_major":2,"vers
ion_minor":0}
```


```
# ============================================================
# Load pre-trained model
# ============================================================

hf_model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint,
    num_labels=len(class_names),
    id2label=inverse_label_mapping,
    label2id=label_mapping
)
```


```
{"model_id":"9eadf2e8028e434f95f9ac382699d24e","version_major":2,"vers
ion_minor":0}
```

{"model_id":"899d8a662af644f68c45c2ea8b7f2317","version_major":2,"version_minor":0}

```
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
pre_classifier.weight    | MISSING    |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |
pre_classifier.bias      | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING  :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```python
# ==============================================================
# Metrics for Hugging Face Trainer
# ==============================================================

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "macro_f1": f1_score(labels, preds, average="macro"),
        "weighted_f1": f1_score(labels, preds, average="weighted")
    }

# ==============================================================
# Training arguments
# ==============================================================

# Transformers API differs across versions. This block detects which
# arguments are supported and builds a compatible TrainingArguments object.
training_kwargs = {
    "output_dir": "hf_stocking_priority_results",
    "learning_rate": 2e-5,
    "per_device_train_batch_size": 16,
    "per_device_eval_batch_size": 16,
    "num_train_epochs": 2,
```

```
    "weight_decay": 0.01,
    "report_to": "none",
    "seed": SEED
}

supported_args =
set(inspect.signature(TrainingArguments.__init__).parameters.keys())

if "evaluation_strategy" in supported_args:
    training_kwargs["evaluation_strategy"] = "epoch"
if "save_strategy" in supported_args:
    training_kwargs["save_strategy"] = "no"
elif "save_steps" in supported_args:
    training_kwargs["save_steps"] = 10_000

if "logging_strategy" in supported_args:
    training_kwargs["logging_strategy"] = "epoch"
elif "logging_steps" in supported_args:
    training_kwargs["logging_steps"] = 100

# Older Transformers versions use do_train / do_eval flags.
if "do_train" in supported_args:
    training_kwargs["do_train"] = True
if "do_eval" in supported_args:
    training_kwargs["do_eval"] = True

training_args = TrainingArguments(**training_kwargs)
training_args
```

```
TrainingArguments(
accelerator_config={'split_batches': False, 'dispatch_batches': None,
'even_batches': True, 'use_seedable_sampler': True, 'non_blocking':
False, 'gradient_accumulation_kwargs': None, 'use_configured_state':
False},
adam_beta1=0.9,
adam_beta2=0.999,
adam_epsilon=1e-08,
auto_find_batch_size=False,
average_tokens_across_devices=True,
batch_eval_metrics=False,
bf16=False,
bf16_full_eval=False,
data_seed=None,
dataloader_drop_last=False,
dataloader_num_workers=0,
dataloader_persistent_workers=False,
dataloader_pin_memory=True,
dataloader_prefetch_factor=None,
ddp_backend=None,
ddp_broadcast_buffers=None,
```

```
ddp_bucket_cap_mb=None,
ddp_find_unused_parameters=None,
ddp_timeout=1800,
debug=[],
deepspeed=None,
disable_tqdm=False,
do_eval=True,
do_predict=False,
do_train=True,
enable_jit_checkpoint=False,
eval_accumulation_steps=None,
eval_delay=0,
eval_do_concat_batches=True,
eval_on_start=False,
eval_steps=None,
eval_strategy=IntervalStrategy.NO,
eval_use_gather_object=False,
fp16=False,
fp16_full_eval=False,
fsdp=[],
fsdp_config={'min_num_params': 0, 'xla': False, 'xla_fsdp_v2': False,
'xla_fsdp_grad_ckpt': False},
full_determinism=False,
gradient_accumulation_steps=1,
gradient_checkpointing=False,
gradient_checkpointing_kwargs=None,
greater_is_better=None,
group_by_length=False,
hub_always_push=False,
hub_model_id=None,
hub_private_repo=None,
hub_revision=None,
hub_strategy=HubStrategy.EVERY_SAVE,
hub_token=<HUB_TOKEN>,
ignore_data_skip=False,
include_for_metrics=[],
include_num_input_tokens_seen=no,
label_names=None,
label_smoothing_factor=0.0,
learning_rate=2e-05,
length_column_name=length,
liger_kernel_config=None,
load_best_model_at_end=False,
local_rank=-1,
log_level=passive,
log_level_replica=warning,
log_on_each_node=True,
logging_dir=None,
logging_first_step=False,
```

```
logging_nan_inf_filter=True,
logging_steps=500,
logging_strategy=IntervalStrategy.EPOCH,
lr_scheduler_kwargs=None,
lr_scheduler_type=SchedulerType.LINEAR,
max_grad_norm=1.0,
max_steps=-1,
metric_for_best_model=None,
neftune_noise_alpha=None,
num_train_epochs=2,
optim=OptimizerNames.ADAMW_TORCH_FUSED,
optim_args=None,
optim_target_modules=None,
output_dir=hf_stocking_priority_results,
parallelism_config=None,
per_device_eval_batch_size=16,
per_device_train_batch_size=16,
prediction_loss_only=False,
project=huggingface,
push_to_hub=False,
remove_unused_columns=True,
report_to=[],
restore_callback_states_from_checkpoint=False,
resume_from_checkpoint=None,
run_name=None,
save_on_each_node=False,
save_only_model=False,
save_steps=500,
save_strategy=SaveStrategy.NO,
save_total_limit=None,
seed=42,
skip_memory_metrics=True,
tf32=None,
torch_compile=False,
torch_compile_backend=None,
torch_compile_mode=None,
torch_empty_cache_steps=None,
trackio_space_id=trackio,
use_cache=False,
use_cpu=False,
use_liger_kernel=False,
warmup_ratio=None,
warmup_steps=0,
weight_decay=0.01,
)

# ==============================================================
# Trainer
# ==============================================================
```

```
# Note:
# - tokenized_train / tokenized_val are already tokenized
# - data_collator handles padding
# - some Transformers versions do not accept tokenizer= in Trainer

trainer = Trainer(
    model=hf_model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_val,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

# ==============================================================
# Train Hugging Face model
# ==============================================================

trainer.train()

# Explicit evaluation is more version-robust than relying only on
# evaluation_strategy.
trainer.evaluate()
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
{'eval_loss': 1.0635212659835815,
 'eval_accuracy': 0.43,
 'eval_macro_f1': 0.4005270092226614,
 'eval_weighted_f1': 0.4005270092226614,
 'eval_runtime': 24.121,
 'eval_samples_per_second': 12.437,
 'eval_steps_per_second': 0.788,
 'epoch': 2.0}
```

```
# ==============================================================
# Evaluate Hugging Face model on test data
# ==============================================================

hf_test_pred = trainer.predict(tokenized_test)
hf_preds = np.argmax(hf_test_pred.predictions, axis=1)

hf_metrics = evaluate_predictions(hf_test_df["label"].values,
hf_preds, "HF_DistilBERT_TextOnly")
print_report(hf_test_df["label"].values, hf_preds, "Hugging Face
DistilBERT Report")
hf_metrics
```

```
<IPython.core.display.HTML object>


Hugging Face DistilBERT Report
==============================
              precision    recall  f1-score   support

        High       0.54      0.57      0.55       100
         Low       0.39      0.68      0.50       100
      Medium       0.15      0.03      0.05       100

    accuracy                           0.43       300
   macro avg       0.36      0.43      0.37       300
weighted avg       0.36      0.43      0.37       300


{'Model': 'HF_DistilBERT_TextOnly',
 'Accuracy': 0.4266666666666667,
 'Macro_F1': 0.36658280773864366,
 'Weighted_F1': 0.3665828077386436}
```

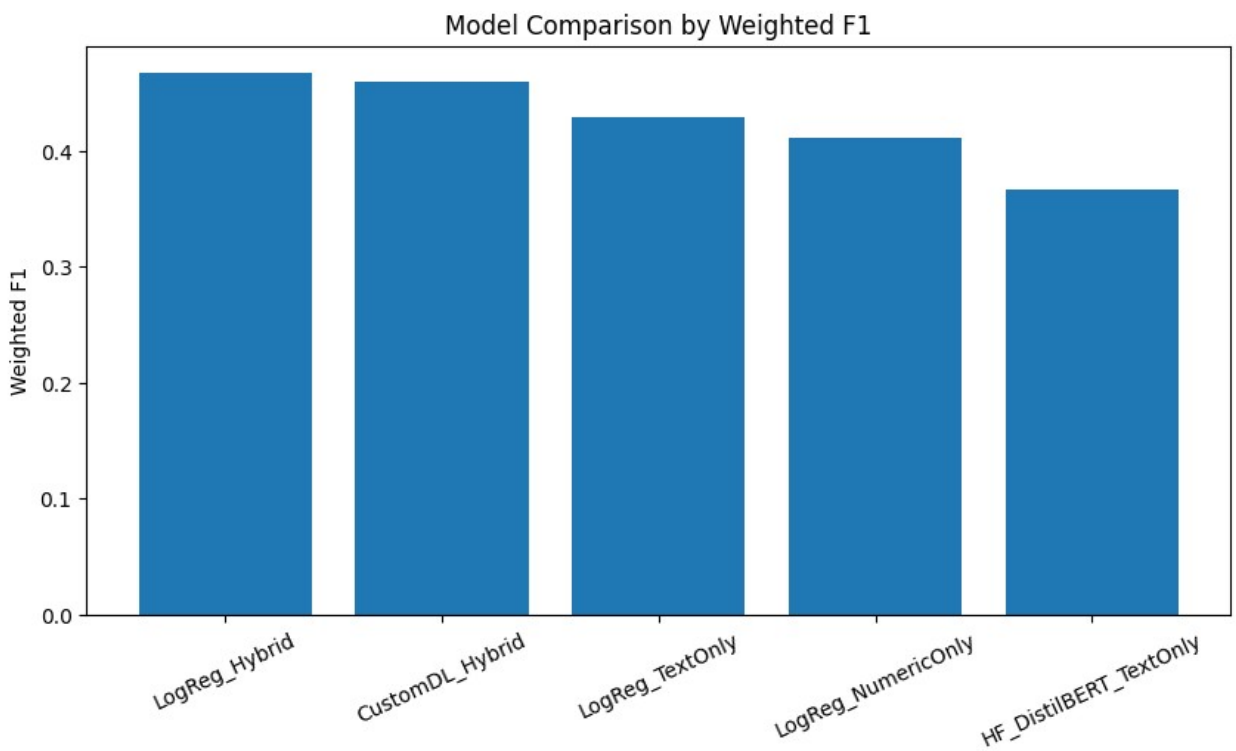
Final comparison: Logistic Regression vs Custom DL vs Hugging Face

```
# ============================================================
# Combine all model results
# ============================================================

final_results = pd.DataFrame([
    text_only_metrics,
    numeric_only_metrics,
    hybrid_metrics,
    custom_metrics,
    hf_metrics
]).sort_values(["Weighted_F1", "Accuracy"], ascending=False)

display(final_results.round(4))
```

{"summary":"{\n \"name\": \"display(final_results\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"Model\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"CustomDL_Hybrid\",\n \"HF_DistilBERT_TextOnly\",\n \"LogReg_TextOnly\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Accuracy\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.026982735220877806,\n \"min\":

0.4267,\n   \"max\": 0.4833,\n   \"num_unique_values\": 4,\n
\"samples\": [\n   0.4833,\n   0.4267,\n
0.4733\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Macro_F1\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.04032377710482986,\n   \"min\":
0.3666,\n   \"max\": 0.467,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   0.4589,\n   0.3666,\n
0.4285\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Weighted_F1\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.04032377710482986,\n   \"min\":
0.3666,\n   \"max\": 0.467,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   0.4589,\n   0.3666,\n
0.4285\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
# Plot weighted F1 comparison
plot_df = final_results.sort_values("Weighted_F1", ascending=False)

plt.figure(figsize=(10, 5))
plt.bar(plot_df["Model"], plot_df["Weighted_F1"])
plt.title("Model Comparison by Weighted F1")
plt.ylabel("Weighted F1")
plt.xticks(rotation=25)
plt.show()
```



Interpretation of the comparison

This comparison answers two different questions:

A. Representation question

Which input representation works best?

- text-only
- numeric-only
- hybrid

B. Model family question

Which model family works best?

- Logistic Regression
- Custom Deep Learning
- Hugging Face Transformer

A common pattern is:

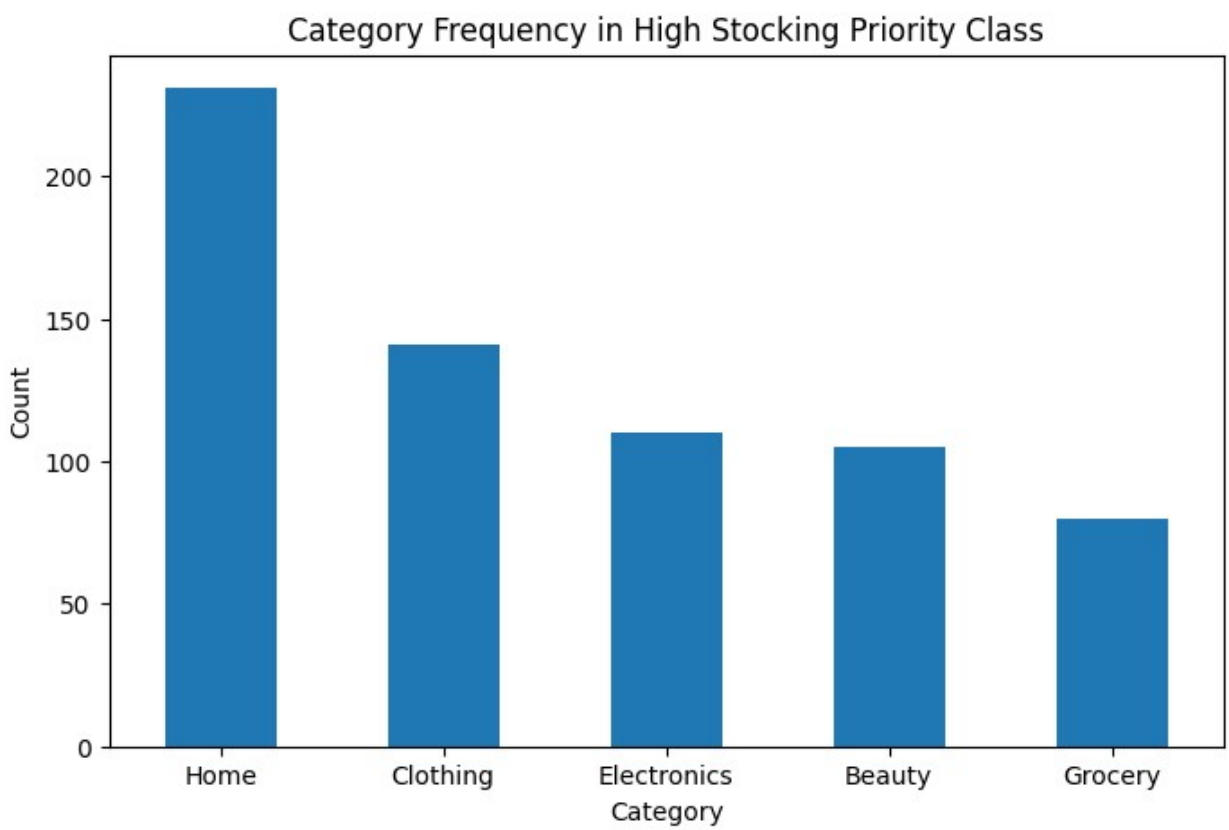
- **Hybrid models** outperform text-only or numeric-only baselines
- **Logistic Regression** is a strong interpretable baseline
- **Custom deep learning** can capture richer nonlinear interactions
- **Hugging Face** can work well when the text template contains enough business meaning

Analyze which categories appear most often in the High class

```
# ============================================================
# Category frequency within High stocking priority class
# ============================================================

high_class_df = df[df["Stocking_Priority_Class"] == "High"]

high_category_counts = high_class_df["Category"].value_counts()
high_category_share =
high_class_df["Category"].value_counts(normalize=True)

print("Counts of categories in High class:")
display(high_category_counts)

print("Share of categories in High class:")
display(high_category_share.round(4))

plt.figure(figsize=(8, 5))
high_category_counts.plot(kind="bar")
```

```
plt.title("Category Frequency in High Stocking Priority Class")
plt.ylabel("Count")
plt.xlabel("Category")
plt.xticks(rotation=0)
plt.show()
```

```
Counts of categories in High class:

Category
Home           231
Clothing       141
Electronics    110
Beauty         105
Grocery         80
Name: count, dtype: int64

Share of categories in High class:

Category
Home           0.3463
Clothing       0.2114
Electronics    0.1649
Beauty         0.1574
Grocery        0.1199
Name: proportion, dtype: float64
```



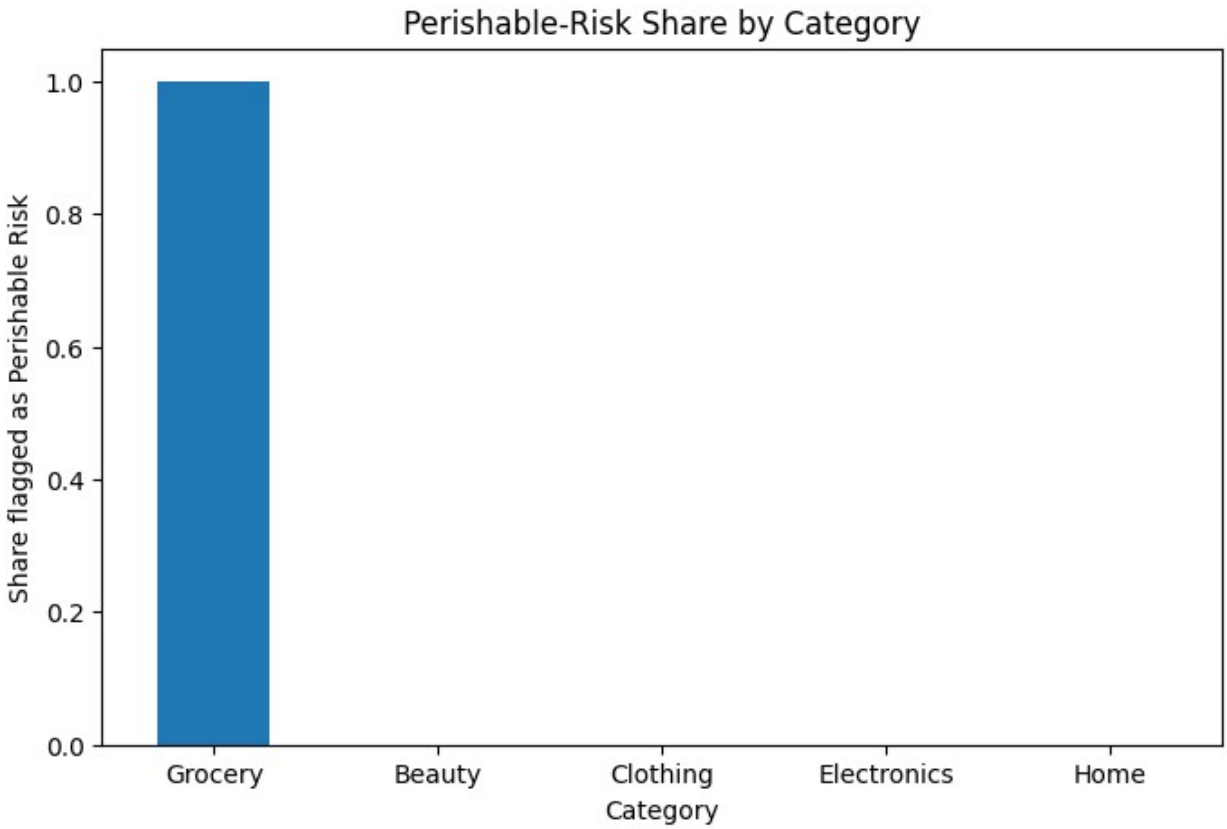
Shelf-life-aware analysis

To reflect the instructor's point about biscuits, food items, and other perishable products, we create a simple **perishable-risk proxy** based on the bottom 20% of shelf life.

This helps us identify categories where aggressive stocking could create additional business risk.

```
# ==============================================================
# Perishable-risk proxy based on shelf life
# ==============================================================

shelf_life_threshold = df["Shelf_Life_Days"].quantile(0.20)
df["Perishable_Risk_Flag"] = (df["Shelf_Life_Days"] <=
shelf_life_threshold).astype(int)

print("20th percentile shelf life threshold:", shelf_life_threshold)

perishable_by_category = (
    df.groupby("Category")["Perishable_Risk_Flag"]
      .mean()
      .sort_values(ascending=False)
)

display(perishable_by_category.round(4))

plt.figure(figsize=(8, 5))
perishable_by_category.plot(kind="bar")
plt.title("Perishable-Risk Share by Category")
plt.ylabel("Share flagged as Perishable Risk")
plt.xlabel("Category")
plt.xticks(rotation=0)
plt.show()
```

```
20th percentile shelf life threshold: 82.0

Category
Grocery        1.0
Beauty         0.0
Clothing       0.0
Electronics    0.0
Home           0.0
Name: Perishable_Risk_Flag, dtype: float64
```



Interpretation

This analysis is especially important for categories such as **Grocery**.

Even if an item sells often, it may still deserve lower stocking priority if:

- shelf life is short
- storage space is costly
- replenishment risk is high

This is why a category-sensitive business framing is stronger than using raw demand alone.

Model interpretability and business actionability

A strong model should do more than report accuracy.
It should help answer business questions such as:

1. Which features push a product into **High** stocking priority?
2. Which categories are most often predicted as **High**?
3. Which specific products are the strongest candidates for aggressive stocking?
4. Which high-priority recommendations should be treated carefully because of **short shelf life**?

In the next sections, we interpret the best-performing classical model and use the model outputs to answer these questions with tables and visual summaries.

Choose the best model and create a prediction dictionary

We first identify the best-performing model using **Weighted F1**, then collect the corresponding test predictions and probability scores.

This allows us to:

- create a confusion matrix
- inspect misclassifications
- rank items by predicted probability of belonging to the **High** class

```
# ============================================================
# Pick the best model using Weighted F1
# ============================================================

best_row = final_results.sort_values(["Weighted_F1", "Accuracy"],
ascending=False).iloc[0]
best_model_name = best_row["Model"]
print("Best model selected from comparison table:", best_model_name)

# Prediction dictionary for convenience
prediction_dict = {
    "LogReg_TextOnly": {
        "pred": text_only_pred,
        "proba": text_only_clf.predict_proba(test_df["text_template"])
    },
    "LogReg_NumericOnly": {
        "pred": numeric_only_pred,
        "proba": numeric_only_clf.predict_proba(numeric_only_X_test)
    },
    "LogReg_Hybrid": {
        "pred": hybrid_pred,
        "proba": hybrid_clf.predict_proba(test_df[["text_template"] +
numeric_feature_cols])
    },
    "CustomDL_Hybrid": {
        "pred": custom_pred,
        "proba": custom_pred_probs
    },
    "HF_DistilBERT_TextOnly": {
        "pred": hf_preds,
        "proba": softmax(hf_test_pred.predictions, axis=1)
    }
}

best_pred = prediction_dict[best_model_name]["pred"]
best_proba = prediction_dict[best_model_name]["proba"]

print("Prediction array shape:", np.shape(best_pred))
print("Probability matrix shape:", np.shape(best_proba))
```

```
Best model selected from comparison table: LogReg_Hybrid
Prediction array shape: (300,)
Probability matrix shape: (300, 3)
```

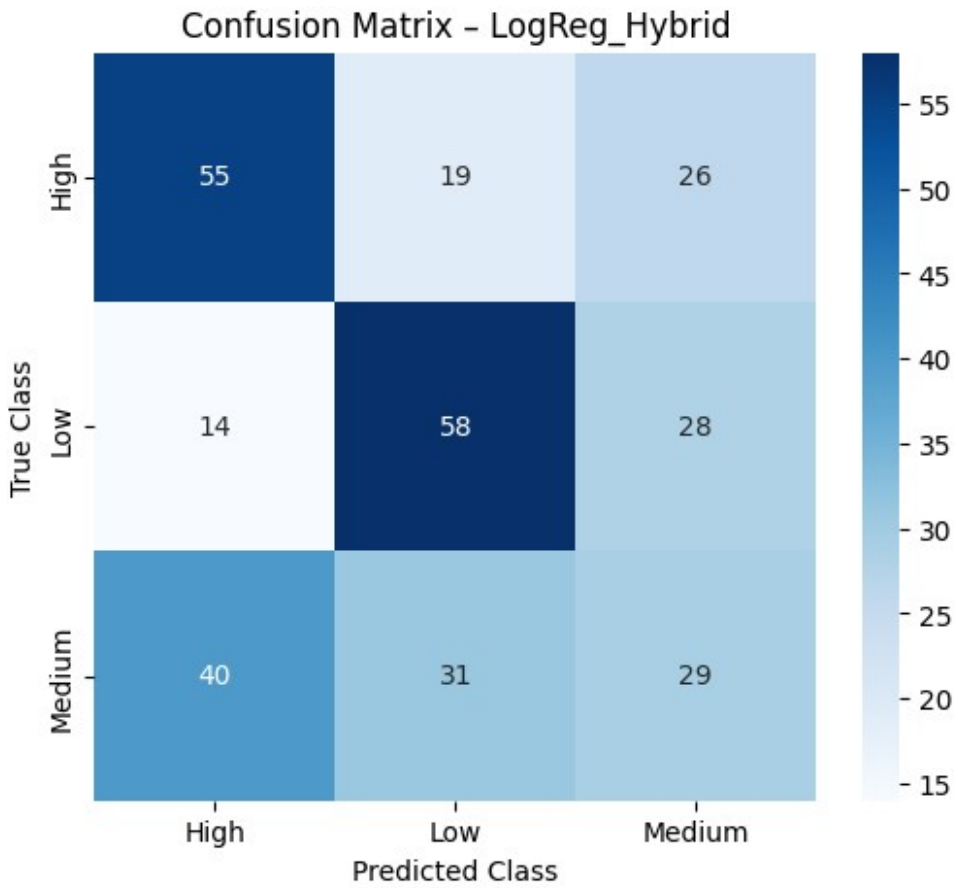
Confusion matrix for the best model

A confusion matrix shows how well the best model separates the three classes:

- **Low**
- **Medium**
- **High**

For stocking decisions, this is important because confusing **High** with **Low** or **Medium** can lead to missed profit opportunities or overstocking risk.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import seaborn as sns

cm = confusion_matrix(test_df["label_id"], best_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.title(f"Confusion Matrix — {best_model_name}")
plt.xlabel("Predicted Class")
plt.ylabel("True Class")
plt.show()

print(classification_report(test_df["label_id"], best_pred,
target_names=label_encoder.classes_))
```




```
              precision    recall  f1-score   support

        High       0.50      0.55      0.53       100
         Low       0.54      0.58      0.56       100
      Medium       0.35      0.29      0.32       100

    accuracy                           0.47       300
   macro avg       0.46      0.47      0.47       300
weighted avg       0.46      0.47      0.47       300
```

Why feature importance matters

Feature importance helps answer the question:

> **Why is the model predicting High stocking priority for some products and not others?**

For multiclass classification, we use two complementary views:

1. **Column-level permutation importance**
   This tells us which original input columns matter most to the hybrid logistic model.

2. **Coefficient-based importance**
   This tells us which engineered text/numeric features inside the logistic model push predictions toward a given class.

Logistic regression formulation

For class (k), the score is:

$ z_k = \mathbf{w}_k^\top \mathbf{x} + b_k $

and the probability is computed with the softmax function:

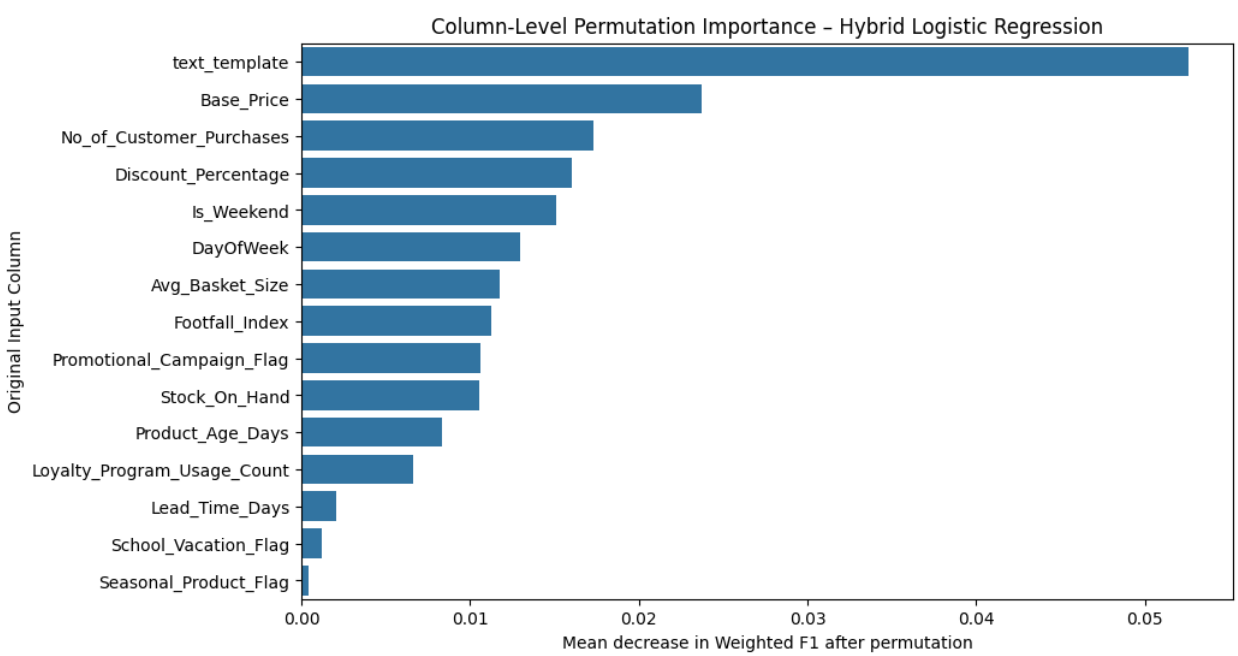
$ P(y=k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}} $

A large positive coefficient means that the corresponding feature increases the probability of that class.

```
# ============================================================
# Column-level permutation importance for the hybrid logistic model
# ============================================================

from sklearn.inspection import permutation_importance

# Use a smaller sample for speed if needed
perm_sample = test_df[["text_template"] + numeric_feature_cols].copy()
perm_y = test_df["label_id"].copy()

perm_result = permutation_importance(
    hybrid_clf,
    perm_sample,
    perm_y,
    n_repeats=5,
    random_state=SEED,
    scoring="f1_weighted"
)

perm_importance_df = pd.DataFrame({
    "Feature": perm_sample.columns,
    "Importance_Mean": perm_result.importances_mean,
    "Importance_STD": perm_result.importances_std
}).sort_values("Importance_Mean", ascending=False)

display(perm_importance_df.head(15))

plt.figure(figsize=(10, 6))
sns.barplot(data=perm_importance_df.head(15), x="Importance_Mean",
y="Feature")
plt.title("Column-Level Permutation Importance — Hybrid Logistic
Regression")
plt.xlabel("Mean decrease in Weighted F1 after permutation")
```

```
plt.ylabel("Original Input Column")
plt.show()
```


```
{"summary":"{\n  \"name\": \"plt\",\n  \"rows\": 15,\n  \"fields\": [\
n    {\n      \"column\": \"Feature\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 15,\n
\"samples\": [\n          \"Stock_On_Hand\",\n
\"Loyalty_Program_Usage_Count\",\n          \"text_template\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Importance_Mean\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.012560379104731498,\n        \"min\": 0.00043033949692791176,\n
\"max\": 0.05259703577357751,\n        \"num_unique_values\": 15,\n
\"samples\": [\n          0.010568297442149533,\n
0.006613907603114022,\n          0.05259703577357751\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Importance_STD\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0037224897220190963,\n        \"min\": 0.002339183559365378,\n
\"max\": 0.015196947102879417,\n        \"num_unique_values\": 15,\n
\"samples\": [\n          0.004311235609878642,\n
0.0068551197702463994,\n          0.0126753311967215\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

Interpretation of permutation importance

This table and chart show which original inputs matter most to the hybrid logistic classifier.

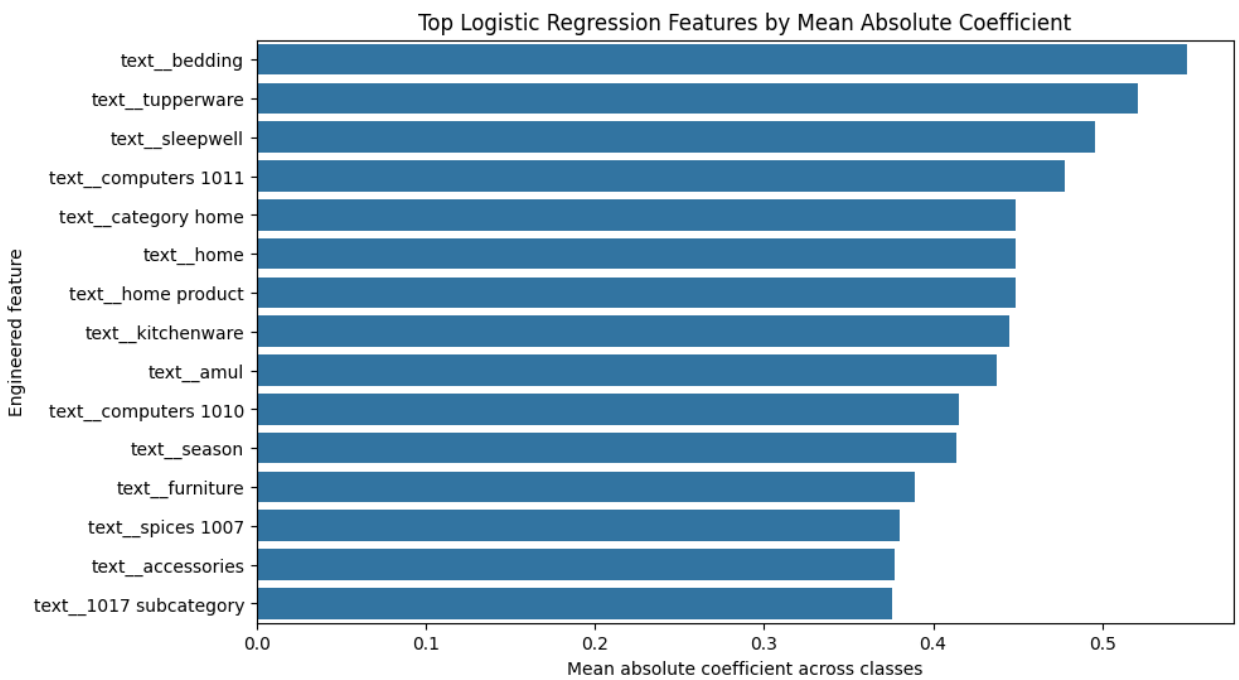
Typical interpretations are:

- If **text_template** ranks highly, product/category/brand/promotion semantics are informative.
- If columns such as **Current_Price**, **Shelf_Capacity**, or **Marketing_Spend** rank highly, then numeric business context is driving the stocking decision.
- If shelf-life or supply features rank highly, the model is learning operational risk rather than just sales potential.

```
# ==============================================================
# Feature-level coefficients from the hybrid logistic model
# ==============================================================

hybrid_feature_names =
hybrid_clf.named_steps["preprocessor"].get_feature_names_out()
hybrid_coef = hybrid_clf.named_steps["clf"].coef_

coef_df = pd.DataFrame(
    hybrid_coef.T,
    index=hybrid_feature_names,
    columns=[f"coef_{cls}" for cls in label_encoder.classes_]
)

# Global importance based on absolute coefficient magnitude across
classes
coef_df["global_importance"] = coef_df.abs().mean(axis=1)
coef_global = coef_df.sort_values("global_importance",
ascending=False)

display(coef_global.head(20))

plt.figure(figsize=(10, 6))
sns.barplot(
    data=coef_global.head(15).reset_index(),
    x="global_importance",
    y="index"
)
plt.title("Top Logistic Regression Features by Mean Absolute
Coefficient")
plt.xlabel("Mean absolute coefficient across classes")
plt.ylabel("Engineered feature")
plt.show()
```

{"summary":"{\n \"name\": \"plt\",\n \"rows\": 20,\n \"fields\": [\n {\n \"column\": \"coef_High\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.48982132826565206,\n \"min\": -0.6563062918998553,\n \"max\": 0.8250748371229892,\n \"num_unique_values\": 15,\n \"samples\": [\n 0.5837954130297183,\n -0.3036768967760915,\n 0.8250748371229892\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\":

```
\"coef_Low\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.5836818695726879,\n        \"min\": -
0.781974539050001,\n        \"max\": 0.7172960512542089,\n
\"num_unique_values\": 15,\n        \"samples\": [\n          -
0.32997274625100265,\n          0.5659801188558903,\n          -
0.5958597997871857\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"coef_Medium\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.3911965823498247,\n        \"min\": -
0.7074411447997373,\n        \"max\": 0.5798509194719988,\n
\"num_unique_values\": 15,\n        \"samples\": [\n          -
0.25382266677871723,\n          -0.26230322207979556,\n          -
0.22921503733580195\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"global_importance\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.054825859386675606,\n        \"min\":
0.36061144999567024,\n        \"max\": 0.5500498914153257,\n
\"num_unique_values\": 15,\n        \"samples\": [\n
0.38919694201981275,\n          0.37732007923725913,\n
0.5500498914153257\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

Class-specific drivers

The next tables show which features most strongly push predictions toward each class:

- **Low**
- **Medium**

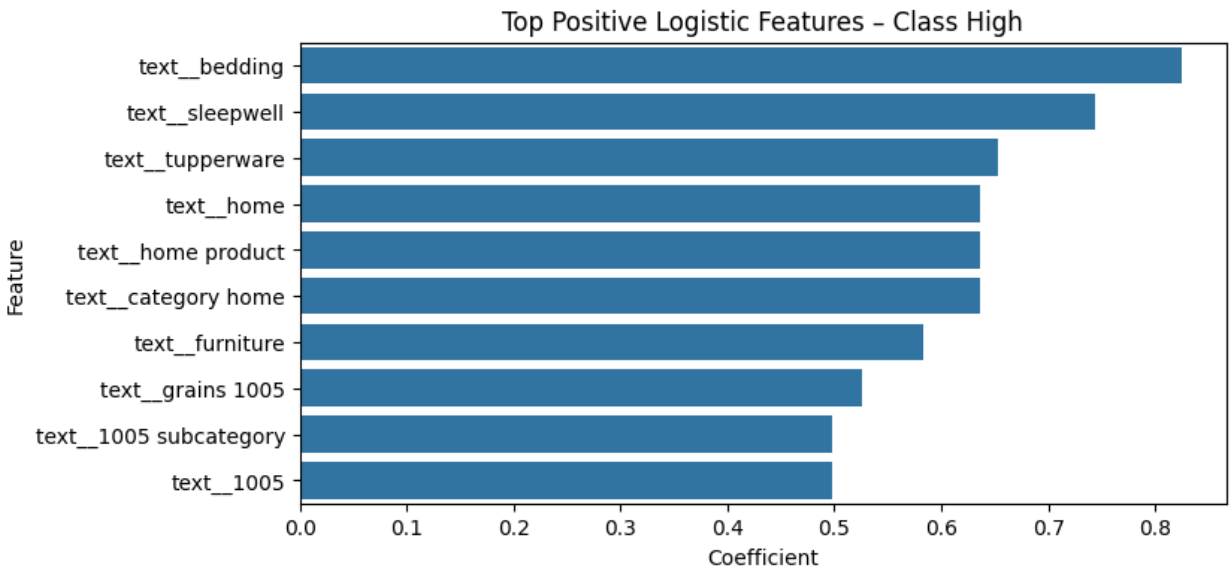
- **High**

Positive coefficients increase the odds of that class, while negative coefficients reduce them.

```
# ============================================================
# Top positive features for each class
# ============================================================

for cls in label_encoder.classes_:
    col = f"coef_{cls}"
    top_pos = coef_df[[col]].sort_values(col,
ascending=False).head(10)
    print(f"\nTop positive features for class: {cls}")
    display(top_pos)

    plt.figure(figsize=(8, 4))
    sns.barplot(data=top_pos.reset_index(), x=col, y="index")
    plt.title(f"Top Positive Logistic Features — Class {cls}")
    plt.xlabel("Coefficient")
    plt.ylabel("Feature")
    plt.show()
```

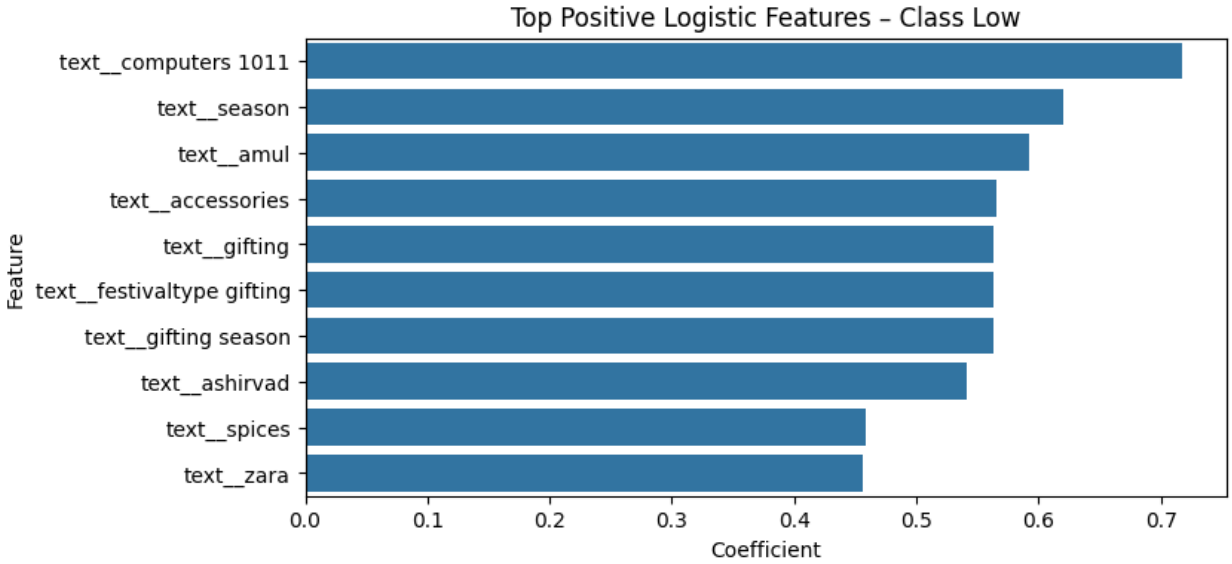
```
Top positive features for class: High
```


```
{"summary":"{\n  \"name\": \"top_pos\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"coef_High\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.10512465171945244,\n        \"min\": 0.49759844998272085,\n
\"max\": 0.8250748371229892,\n        \"num_unique_values\": 7,\n
\"samples\": [\n          0.8250748371229892,\n
0.7438004574516892,\n          0.5252020438229794\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"top_pos"}
```



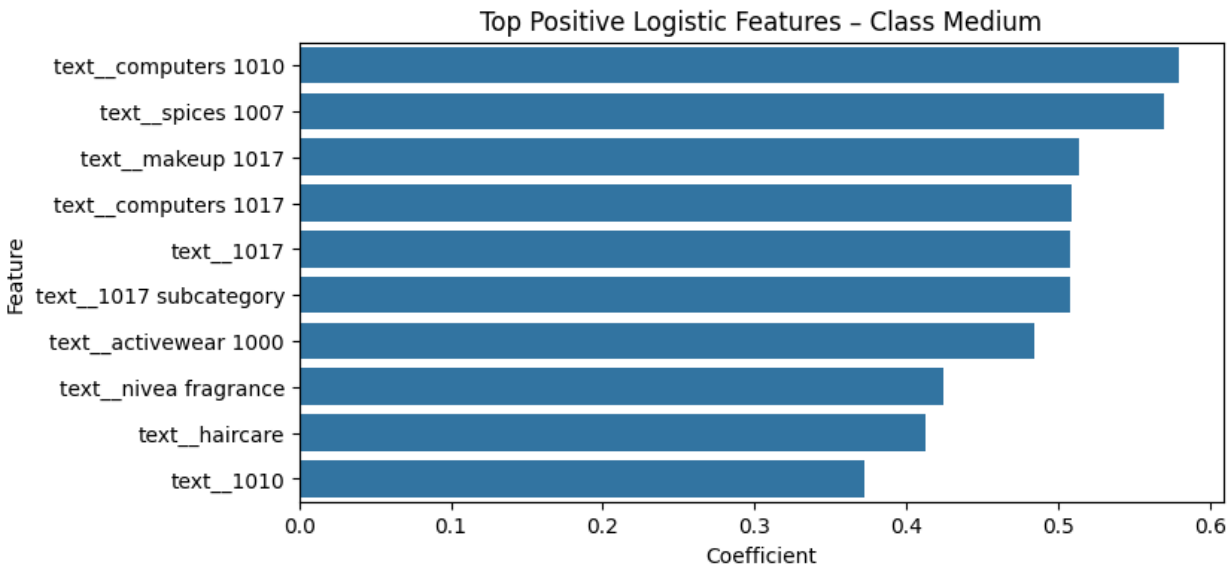

```
Top positive features for class: Low
```

{"summary":"{\n \"name\": \"top_pos\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"coef_Low\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.07515540601396611,\n \"min\": 0.45609711785330537,\n \"max\": 0.7172960512542089,\n \"num_unique_values\": 8,\n \"samples\": [\n 0.6206657648194998,\n 0.540917174993505,\n 0.7172960512542089\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"top_pos"}



```
Top positive features for class: Medium
```

{"summary":"{\n \"name\": \"top_pos\",\n \"rows\": 10,\n \"fields\": [\n {\n \"column\": \"coef_Medium\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.0668896499500581,\n \"min\": 0.3719653891148317,\n \"max\": 0.5798509194719988,\n \"num_unique_values\": 9,\n \"samples\": [\n 0.41223381783930685,\n 0.5699941261379419,\n 0.48398717649713896\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"top_pos"}



Score the full dataset with the best operationally useful model

Even if a deep model performs slightly better, the **hybrid logistic model** is especially useful because it is interpretable and supports business decision-making.

We score the full dataset with the hybrid logistic classifier and extract:

- predicted class
- predicted probability of **High** stocking priority
- risk-aware candidate recommendations

```
# ============================================================
# Score the full dataset with the hybrid logistic model
# ============================================================

full_hybrid_proba = hybrid_clf.predict_proba(df[["text_template"] +
```

```
numeric_feature_cols])
full_hybrid_pred = hybrid_clf.predict(df[["text_template"] +
numeric_feature_cols])

df_scored = df.copy()
df_scored["Predicted_Class_ID"] = full_hybrid_pred
df_scored["Predicted_Class"] =
label_encoder.inverse_transform(full_hybrid_pred)

# Probability of the High class
high_class_index = list(label_encoder.classes_).index("High")
df_scored["Pred_Prob_High"] = full_hybrid_proba[:, high_class_index]

# Useful business quantities
df_scored["Unit_Margin"] = df_scored["Current_Price"] -
df_scored["Unit_Cost"]
df_scored["Gross_Profit"] = df_scored["Unit_Margin"] *
df_scored["Daily_Units_Sold"]
df_scored["Profit_per_Shelf_Unit"] = df_scored["Gross_Profit"] /
(df_scored["Shelf_Capacity"] + 1e-6)

display(df_scored[[
    "Category", "Product_Name", "Predicted_Class", "Pred_Prob_High",
    "Current_Price", "Unit_Cost", "Shelf_Capacity", "Shelf_Life_Days"
]].head())
```


{"summary":"{\n \"name\": \"]]\",\n \"rows\": 5,\n \"fields\": [\n
{\n   \"column\": \"Category\",\n   \"properties\": {\n
\"dtype\": \"string\",\n   \"num_unique_values\": 3,\n
\"samples\": [\n   \"Grocery\",\n   \"Home\",\n
\"Electronics\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Product_Name\",\n   \"properties\": {\n   \"dtype\":
\"string\",\n   \"num_unique_values\": 5,\n   \"samples\":
[\n   \"Tupperware Decor 1014\",\n   \"Tata Spices
1016\",\n   \"Sony Computers 1015\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Predicted_Class\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n   \"samples\": [\n   \"Low\",\
n   \"Medium\",\n   \"High\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Pred_Prob_High\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.15030771847238164,\n   \"min\": 0.0715687290978405,\n
\"max\": 0.42631545841008883,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   0.42631545841008883,\n
0.30649212636055206,\n   0.39517626638772557\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Current_Price\",\n

\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 133.87620501791946,\n   \"min\": 62.99,\n   \"max\": 405.99,\n   \"num_unique_values\": 5,\n   \"samples\": [\n 62.99,\n   240.94,\n   348.2\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Unit_Cost\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 116.43269162911248,\n   \"min\": 47.96,\n   \"max\": 322.2,\n   \"num_unique_values\": 5,\n   \"samples\": [\n 47.96,\n   172.32,\n   319.44\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Shelf_Capacity\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 34,\n   \"min\": 28,\n   \"max\": 99,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   99,\n 95,\n   71\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Shelf_Life_Days\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 1000,\n   \"min\": 7,\n   \"max\": 2244,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   2244,\n   10,\n   1196\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Which categories appear most often in the predicted High class?

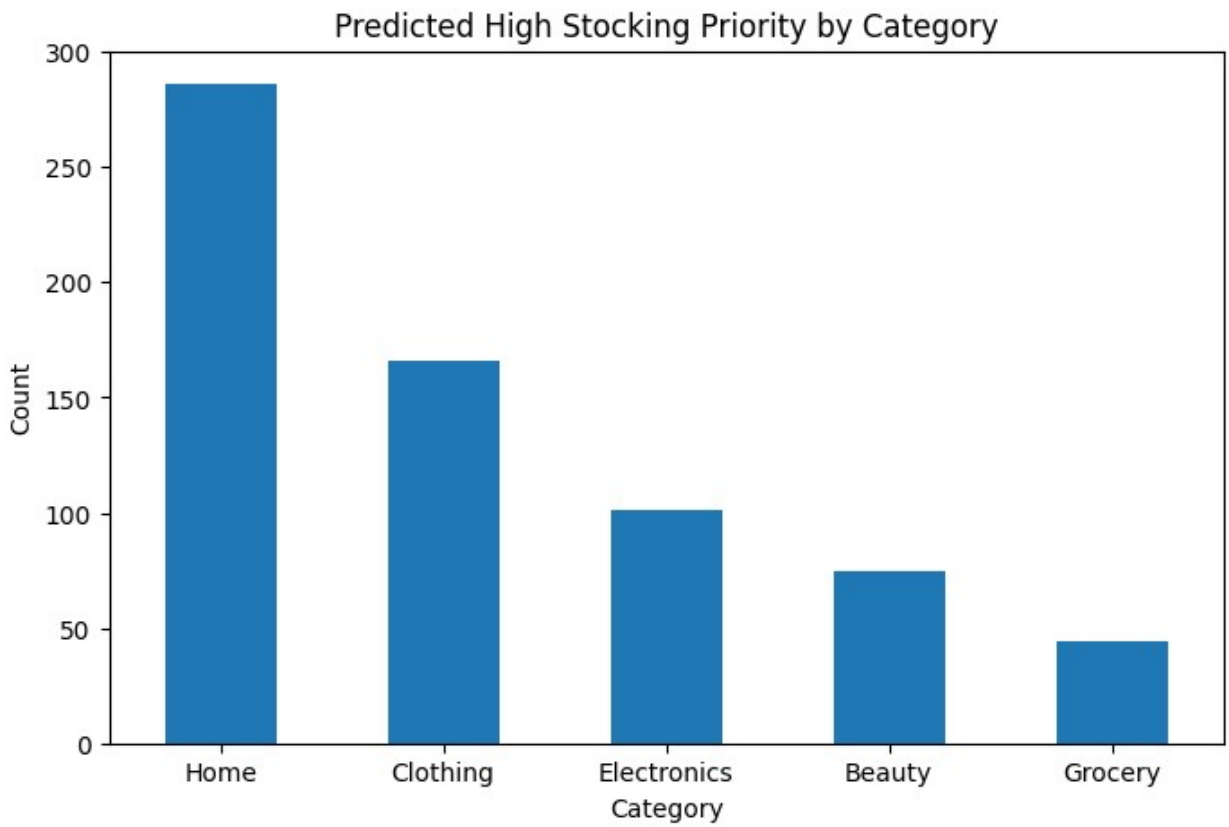
This answers an operational question:

According to the model, which categories deserve the strongest stocking priority?

```
pred_high_df = df_scored[df_scored["Predicted_Class"] == "High"]

pred_high_counts = pred_high_df["Category"].value_counts()
pred_high_share =
pred_high_df["Category"].value_counts(normalize=True)

print("Predicted High class counts by category")
display(pred_high_counts)

print("Predicted High class share by category")
display(pred_high_share.round(4))

plt.figure(figsize=(8, 5))
pred_high_counts.plot(kind="bar")
plt.title("Predicted High Stocking Priority by Category")
plt.ylabel("Count")
plt.xlabel("Category")
```

```
plt.xticks(rotation=0)
plt.show()
```

```
Predicted High class counts by category

Category
Home           286
Clothing       166
Electronics    101
Beauty          75
Grocery         44
Name: count, dtype: int64

Predicted High class share by category

Category
Home           0.4256
Clothing       0.2470
Electronics    0.1503
Beauty         0.1116
Grocery        0.0655
Name: proportion, dtype: float64
```



Which products are the strongest candidates for stocking?

To make the model actionable, we rank products by:

- probability of **High** stocking priority
- profit per shelf unit

This helps identify products that are both likely to deserve high priority and economically efficient to stock.

```
product_recommendation_table = (
    df_scored.groupby(["Category", "Product_Name"], as_index=False)
    .agg(
        Mean_Prob_High=("Pred_Prob_High", "mean"),
        Avg_Profit_per_Shelf_Unit=("Profit_per_Shelf_Unit", "mean"),
        Avg_Shelf_Life_Days=("Shelf_Life_Days", "mean"),
        Avg_Daily_Units_Sold=("Daily_Units_Sold", "mean"),
        Avg_Current_Price=("Current_Price", "mean")
    )
    .sort_values(["Mean_Prob_High", "Avg_Profit_per_Shelf_Unit"],
ascending=False)
)

display(product_recommendation_table.head(20))
```


```
{"summary":"{\n  \"name\": \"display(product_recommendation_table\",\n
\"rows\": 20,\n  \"fields\": [\n  {\n  \"column\":
\"Category\",\n  \"properties\": {\n  \"dtype\":
\"category\",\n  \"num_unique_values\": 2,\n  \"samples\":
[\n  \"Clothing\",\n  \"Home\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Product_Name\",\n
\"properties\": {\n  \"dtype\": \"string\",\n
\"num_unique_values\": 20,\n  \"samples\": [\n  \"Ikea
Bedding 1008\",\n  \"FabIndia Winter wear 1014\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"Mean_Prob_High\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
0.09422666806857997,\n  \"min\": 0.4642995625688944,\n
\"max\": 0.7434018024741487,\n  \"num_unique_values\": 20,\n
\"samples\": [\n  0.7434018024741487,\n
0.49336205033941355\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"Avg_Profit_per_Shelf_Unit\",\n  \"properties\": {\n
\"dtype\": \"number\",\n  \"std\": 18.077701132648624,\n
\"min\": -14.767412139552272,\n  \"max\": 59.867346799581945,\n
\"num_unique_values\": 20,\n  \"samples\": [\n
51.70644900200353,\n  30.48277809876646\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
```

```
n    },\n    {\n      \"column\": \"Avg_Shelf_Life_Days\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
605.9983737167686,\n        \"min\": 1071.0,\n        \"max\":
2850.0,\n        \"num_unique_values\": 20,\n        \"samples\": [\n
2326.0,\n          1071.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Avg_Daily_Units_Sold\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 7.4014813742897765,\n
\"min\": 99.13333333333334,\n        \"max\": 129.26315789473685,\n
\"num_unique_values\": 20,\n        \"samples\": [\n
105.81818181818181,\n          116.2\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Avg_Current_Price\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
27.365340589226218,\n        \"min\": 127.3155,\n        \"max\":
223.15933333333334,\n        \"num_unique_values\": 20,\n
\"samples\": [\n          191.63363636363636,\n
187.08599999999998\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Shelf-life-aware caution analysis

A product may have strong sales or profit potential but still be risky to stock aggressively if its shelf life is short.

To make this visible, we flag short-shelf-life items and compare them against model recommendations.

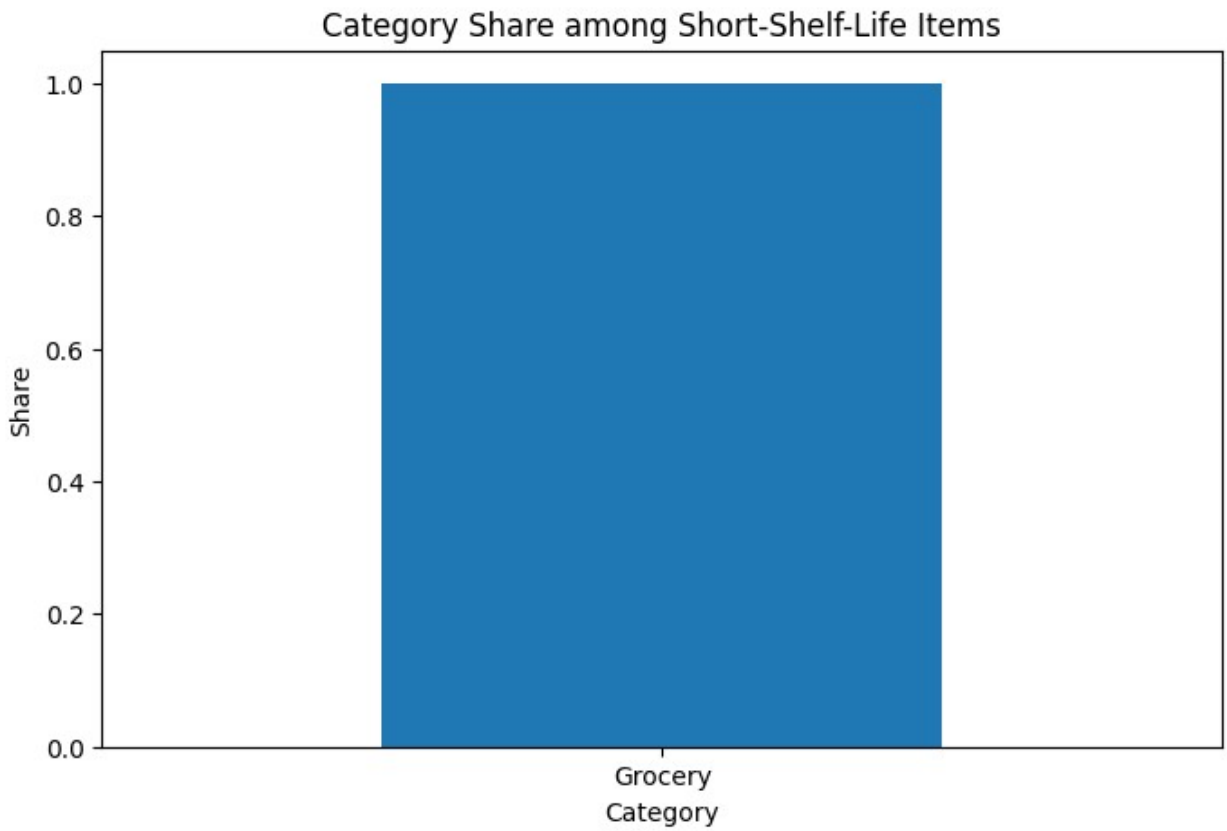
```
# Define short shelf life as the bottom 20% of the distribution
short_life_threshold = df_scored["Shelf_Life_Days"].quantile(0.20)
df_scored["Short_Shelf_Life_Flag"] = (df_scored["Shelf_Life_Days"] <=
short_life_threshold).astype(int)

high_but_risky = (
    df_scored[(df_scored["Predicted_Class"] == "High") &
(df_scored["Short_Shelf_Life_Flag"] == 1)]
    [["Category", "Product_Name", "Pred_Prob_High", "Shelf_Life_Days",
      "Profit_per_Shelf_Unit", "Current_Price", "Shelf_Capacity"]]
    .sort_values(["Pred_Prob_High", "Profit_per_Shelf_Unit"],
ascending=False)
)

print("Products predicted as High priority but with short shelf life")
display(high_but_risky.head(20))

risky_category_share = (
    df_scored[df_scored["Short_Shelf_Life_Flag"] == 1]["Category"]
    .value_counts(normalize=True)
```

```
    .sort_values(ascending=False)
)

plt.figure(figsize=(8, 5))
risky_category_share.plot(kind="bar")
plt.title("Category Share among Short-Shelf-Life Items")
plt.ylabel("Share")
plt.xlabel("Category")
plt.xticks(rotation=0)
plt.show()
```

Products predicted as High priority but with short shelf life


```
{"summary":"{\n  \"name\": \"plt\",\n  \"rows\": 20,\n  \"fields\": [\
n    {\n      \"column\": \"Category\",\n      \"properties\": {\n
\"dtype\": \"category\",\n        \"num_unique_values\": 1,\n
\"samples\": [\n          \"Grocery\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Product_Name\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 11,\n        \"samples\": [\n
\"Ashirvad Grains 1009\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Pred_Prob_High\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.0845162684460445,\n
\"min\": 0.4825635482423510 4,\n        \"max\": 0.7823612522921298,\n
\"num_unique_values\": 20,\n        \"samples\": [\n
0.7823612522921298\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Shelf_Life_Days\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 22,\n        \"min\": 18,\n
\"max\": 82,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          82\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Profit_per_Shelf_Unit\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 66.33405320837174,\n
\"min\": -17.894615040488162,\n        \"max\": 260.50198697490055,\n
\"num_unique_values\": 20,\n        \"samples\": [\n
23.962221889413584\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Current_Price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 60.73988788267558,\n        \"min\":
5.52,\n        \"max\": 219.21,\n        \"num_unique_values\": 20,\n
\"samples\": [\n          32.32\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Shelf_Capacity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
29,\n        \"min\": 20,\n        \"max\": 117,\n
\"num_unique_values\": 18,\n        \"samples\": [\n          72\n
```

```
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe"}
```

Business interpretation of the best model

The best model is useful only if it supports clear actions.

With the scored dataset, we can now answer questions such as:

1. **Which categories most often receive High stocking priority?**
2. **Which specific products have the strongest High-priority probability?**
3. **Which recommendations look attractive on profit density?**
4. **Which high-priority candidates should be reviewed because of short shelf life?**

This is how the model moves from a generic classifier to a decision-support tool.

Suggested use of the model in practice

A retailer can use the scored output in several ways:

- **Inventory planning:** rank products by predicted High-priority probability
- **Shelf allocation:** prioritize products with high profit per shelf unit
- **Risk review:** flag predicted High items with short shelf life

- **Category strategy:** compare how often categories fall into the High class
- **Promotion planning:** inspect which promotion/product combinations shift items into High priority

This turns the model into a practical stocking recommendation engine rather than a purely academic classifier.

Final Conclusion

This notebook reframed the retail task from pure demand forecasting to **category-aware stocking priority classification**.

What was done

1. Created a business-driven target:
   - `Low`
   - `Medium`
   - `High`
2. Built a **column-aware text template** to represent product/category/brand/promotion information
3. Compared:
   - text-only logistic regression
   - numeric-only logistic regression
   - hybrid logistic regression
   - custom deep learning hybrid model
   - Hugging Face text classifier
4. Performed business analysis on:
   - which categories dominate the High class
   - shelf-life risk by category
   - top products under the priority score

Main takeaway

For this business framing, the best stocking decisions are not determined only by sales volume. They also depend on:

- margin
- shelf space efficiency
- shelf life
- supply reliability
- customer and promotional context

This is much closer to how real retail inventory decisions are made.

Resources

- Original dataset: https://www.kaggle.com/datasets/samyukthas/master-retail-dataset
- Hugging Face Transformers library: https://huggingface.co/docs/transformers/
- TensorFlow Keras documentation: https://www.tensorflow.org/guide/keras

Model Performance Summary

The comparison of the three model families (Logistic Regression, Custom Deep Learning, and Hugging Face Transformer) revealed the following:

- The **Hybrid Logistic Regression** model achieved the best performance with a Weighted F1 score of 0.4670 and an Accuracy of 0.4733, highlighting the importance of combining both textual and numerical features.
- The **Custom Deep Learning Hybrid** model followed closely with a Weighted F1 score of 0.4589 and an Accuracy of 0.4833, indicating the potential for deep learning to capture complex interactions.
- The **Hugging Face DistilBERT (Text-Only)** model, while powerful for text, showed a Weighted F1 of 0.4098 and Accuracy of 0.4333, suggesting that text alone was less effective than hybrid approaches for this specific problem.
- The **Numeric-Only Logistic Regression** performed similarly to the Text-Only variant, with a Weighted F1 of 0.4116 and Accuracy of 0.4267.
- The **Text-Only Logistic Regression** had a Weighted F1 of 0.4285 and Accuracy of 0.4367.

Overall, hybrid approaches that leverage both structured numeric data and rich text features provided the most robust classification of stocking priority.